1.1数据结构的基本概念

基本概念和术语

- 数据：数据是信息的载体，是描述客观事物属性的数、字符及所有能输入到计算机中并被计算机程序识别和处理的符号的集合
- 数据元素：数据元素是数据的基本单位，通常作为一个整体进行考虑和处理
- 数据对象：数据对象是具有相同性质的数据元素的集合,是数据的一个子集
- 数据类型：数据类型是一个值的集合和定义在此集合上的一组操作的总称
  - 分类
    - 原子类型：其值不可再分的数据类型
    - 结构类型：其值可以再分解为若干成分(分量)的数据类型
    - 抽象数据类型：抽象数据组织及与之相关的操作
- 数据结构：数据结构是相互之间存在一种或多种特定关系的数据元素的集合
  - 数据结构包括三方面
    - 逻辑结构
    - 存储结构
    - 数据的运算

数据结构三要素

数据的逻辑结构

- 概述
  - 逻辑结构是指数据元素之间的逻辑关系，即从逻辑关系上描述数据
  - 与数据的存储无关， 是独立于计算机的
- 分类
  - 线性结构
    - 线性表
    - 栈
    - 队列
    - 数组
  - 非线性结构
    - 集合
    - 树
    - 图

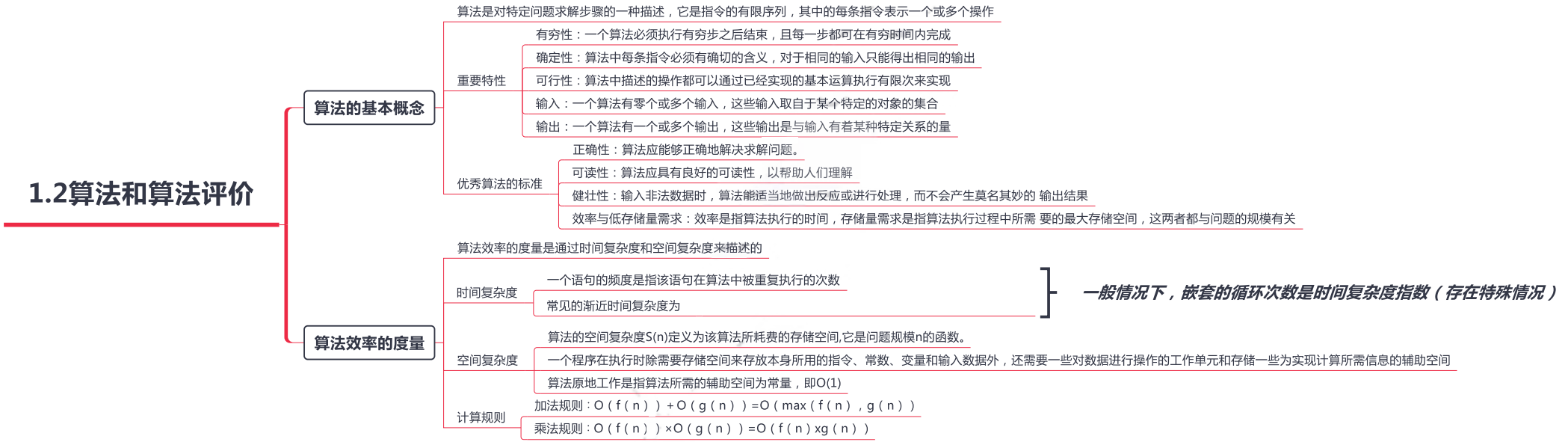
数据的存储结构

- 概述
  - 存储结构是指数据结构在计算机中的表示(又称映像)，也称物理结构
  - 数据元素的表示和关系的表示
  - 存储结构是用计算机语言实现的逻辑结构，它依赖于计算机语言
- 分类
  - 顺序存储
    - 把逻辑上相邻的元素存储在物理位置上也相邻的存储单元中，元素之间的关系由存储单元的邻接关系来体现
    - 优点：可以实现随机存取，每个元素占用最少的存储空间
    - 缺点：只能使用相邻的一整块存储单元，因此可能产生较多的外部碎片
  - 链式存储
    - 要求逻辑上相邻的元素在物理位置上也相邻，借助指示元素存储地址的指 针来表示元素之间的逻辑关系
    - 优点：不会出现碎片现象，能充分利用所有存储单元
    - 缺点：每个元素因存储指针而占用额外的存储空间，且只能实现顺序存取
  - 索引存储
    - 在存储元素信息的同时，还建立附加的索引表
    - 优点：检索速度快
    - 缺点
      - 附加的索引表额外 占用存储空间
      - 增加和删除数据时也要修改索引表，会花费较多的时间
  - 散列存储
    - 根据元素的关键字直接计算出该元素的存储地址，又称哈希（Hash）存储
    - 优点：检索、增加和删除结点的操作都很快
    - 缺点：若散列函数不好，则可能出现元素存储单元的冲突，而解决冲突会增加时间和空间开销

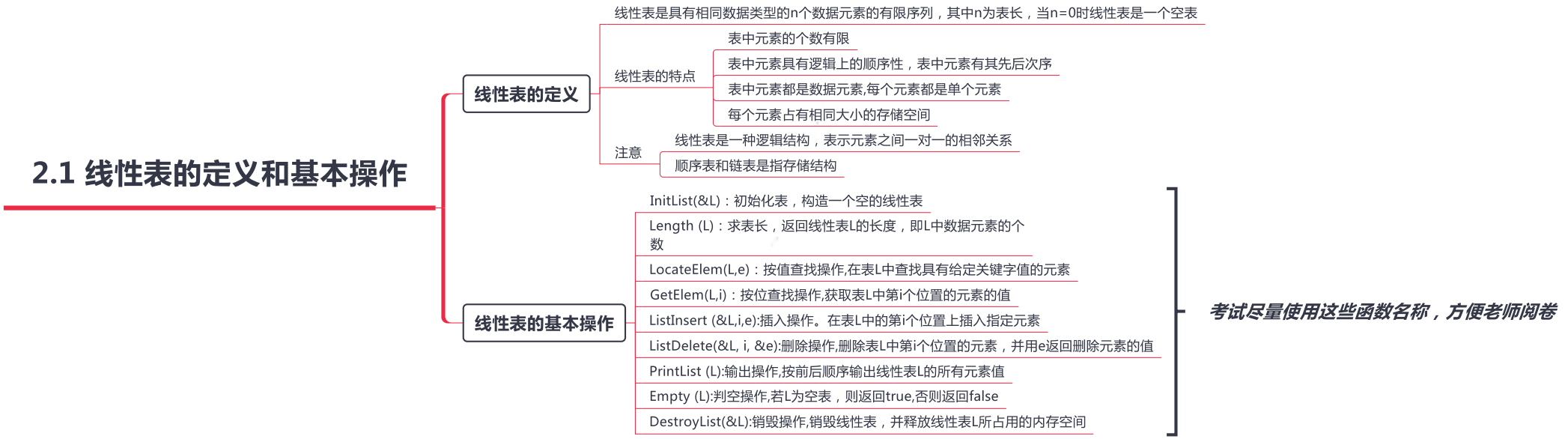
数据的运算

- 施加在数据上的运算包括运算的定义和实现。
- 运算的定义是针对逻辑结构的，指出运算的功能
- 运算的实现是针对存储结构的，指出运算的具体操作步骤

1.2算法和算法评价


算法的基本概念
算法是对特定问题求解步骤的一种描述，它是指令的有限序列，其中的每条指令表示一个或多个操作
重要特性
有穷性：一个算法必须执行有穷步之后结束，且每一步都可在有穷时间内完成
确定性：算法中每条指令必须有确切的含义，对于相同的输入只能得出相同的输出
可行性：算法中描述的操作都可以通过已经实现的基本运算执行有限次来实现
输入：一个算法有零个或多个输入，这些输入取自于某个特定的对象的集合
输出：一个算法有一个或多个输出，这些输出是与输入有着某种特定关系的量
优秀算法的标准
正确性：算法应能够正确地解决求解问题。
可读性：算法应具有良好的可读性，以帮助人们理解
健壮性：输入非法数据时，算法能适当地做出反应或进行处理，而不会产生莫名其妙的 输出结果
效率与低存储量需求：效率是指算法执行的时间，存储量需求是指算法执行过程中所需 要的最大存储空间，这两者都与问题的规模有关
算法效率的度量
算法效率的度量是通过时间复杂度和空间复杂度来描述的
时间复杂度
一个语句的频度是指该语句在算法中被重复执行的次数
常见的渐近时间复杂度为
一般情况下，嵌套的循环次数是时间复杂度指数（存在特殊情况）
空间复杂度
算法的空间复杂度S(n)定义为该算法所耗费的存储空间,它是问题规模n的函数。
一个程序在执行时除需要存储空间来存放本身所用的指令、常数、变量和输入数据外，还需要一些对数据进行操作的工作单元和存储一些为实现计算所需信息的辅助空间
算法原地工作是指算法所需的辅助空间为常量，即O(1)
计算规则
加法规则：O(f(n))+O(g(n))=O(max(f(n),g(n)))
乘法规则：O(f(n))×O(g(n))=O(f(n)xg(n))

2.1 线性表的定义和基本操作



2.2 线性表的顺序表示

顺序表的定义

线性表的顺序存储又称顺序表
用一组地址连续的存储单元依次存储线性表中的数据元素
逻辑上相邻的两个元素在物理位置上也相邻

一维数组空间分配
- 静态分配：数组的大小和空间已经固定，一旦空间占满，再加入新的数据将会产生溢出，程序就会崩溃
- 动态分配：存储数组的空间是在程序执行过程中通过动态存储分配语句分配的
  - 一旦数据空间占满，就另外开辟一块更大的存储空间，用以替换原来的存储空间

注意：动态分配仍然是顺序存储结构，物理结构没有变化，依然是随机存取方式,只是分配的空间大小可以在运行时决定

特点
- 随机访问，即通过首地址和元素序号可在时间O(1)内找到指定的元素
- 存储密度高，每个结点只存储数据元素　　与链表形成对比
- 逻辑上相邻的元素物理上也相邻，当执行插入和删除操作时，需要移动大量元素

顺序表上基本操作的实现

插入操作
- 由于顺序表的中元素的物理位置是相邻的，所以当插入新元素的时候就需要对表中的元素进行整体移动

实现情况(长度为n)
- 最好情况：在表尾插入(i = n+1),元素后移语句将不执行，时间复杂度为O(1)
- 最坏情况：在表头插入(i=1),元素后移语句将执行n次，时间复杂度为0(n)

平均情况
- 假设pi （ pi = 1/(n+1))是在第i个位置上插入一个结点的概率
- 则在长度为n的线性表中插入一个结点时，移动节点的平均次数为 $\sum_{i=1}^{n+1} p_i(n-i+1)=\sum_{i=1}^{n+1}\frac{1}{n+1}(n-i+1)=\frac{1}{n+1}\sum_{i=1}^{n+1}(n-i+1)=\frac{1}{n+1}\frac{n(n+1)}{2}=\frac{n}{2}$
- 时间复杂度为 O(n)

删除操作
- 即为移动元素，对想要删除的元素进行覆盖
- 最好情况：删除表尾元素(i=n)，无须移动元素，时间复杂度为O(1)
- 最坏情况：删除表头元素(i=1),需移动除第一个元素外的所有元素，时间复杂度为O(n)

平均情况
- 假设pi （ pi = 1/n)是在第i个位置上删除一个结点的概率
- 则在长度为n的线性表中删除一个结点时，删除节点的平均次数为 $\sum_{i=1}^{n} p_i(n-i)=\sum_{i=1}^{n}\frac{1}{n}(n-i)=\frac{1}{n}\sum_{i=1}^{n}(n-i)=\frac{1}{n}\frac{n(n-1)}{2}=\frac{n-1}{2}$
- 时间复杂度为 O(n)

按值查找(顺序查找)
- 最好情况:查找的元素就在表头,仅需比较一次,时间复杂度为 O(1)
- 最坏情况:查找的元素在表尾（或不存在）时,需要比较n次,时间复杂度为O(n)

平均情况
- 假设pi(pi=1/n ）是查找的元素在第i（ 1<=i<=L.1ength ）个位置上的概率
- 则在长度为n的线性表中查找值为e的元素所需比较的平均次数为 $\sum_{i=1}^{n} p_i \times i=\sum_{i=1}^{n}\frac{1}{n}\times i=\frac{1}{n}\frac{n(n+1)}{2}=\frac{n+1}{2}$
- 时间复杂度为O(n)

2.3线性表的链式表示(上)

背景

- 顺序表
  - 优点：可以随时存取表中的任意一个元素
  - 缺点：插入和删除操作需要移动大量元素
- 链式存储线性表
  - 优点：不需要使用地址连续的存储单元，插入和删除操作不需要移动元素，而只需修改指针
  - 缺点：失去顺序表可随机存取的优点

实现方式

- 带头结点　空表判断：L==NULL。代码书写不便
- 不带头结点　空表判断：L->next==NULL。写代码更方便

单链表的定义

线性表的链式存储又称单链表，它是指通过一组任意的存储单元来存储线性表中的数据元素

对每个链表结点，除存放 元素自身的信息外，还需要存放一个指向其后继的指针

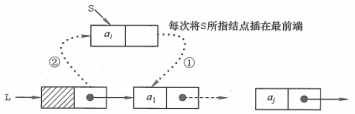
- 结构
  - data：数据域,存放数据元素
  - next ：指针域,存放其后继结点的地址
- 优点　解决顺序表需要大量连续存储单元的缺点
- 缺点
  - 单链表附加指针域，浪费存储空间
  - 查找某个特定的结点时，需要从表头开始遍历，依次查找

单链表上基本操作的实现

采用头插法建立单链表

从一个空表开始,生成新结点,并将读取到的数据存放到新结点的数据域中,然后将新结点插入到当前链表的表头,即头结点之后

- 示意图

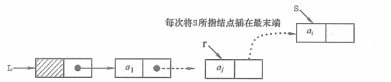


- 时间复杂度
  - 每个结点插入的时间复杂度为O(1)
  - 设单链表长为n,则总时间复杂度为O(n)
- 优点：算法实现简单
- 缺点：生成的链表中结点的次序和输入数据的顺序不一致　可以利用这个特点进行链表转置

采用尾插法建立单链表

该方法将新结点插入到当前链表的表尾,为此必须增加一个尾指针 r,使其始终指向当前链表的尾结点

- 示意图

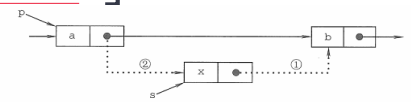


按序号查找结点值　时间复杂度O(n)

按值查找表结点　时间复杂度O(n)

链表的本身特点原因，查找只能依次遍历查找

插入结点操作

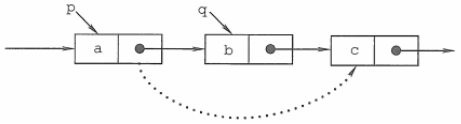


- 示意图
- 实现代码段
  - p=GetElem(L,i-1);
  - s->next=p->next;
  - p->next=s;
- 若在给定的节点下进行插入，则时间复杂度为O(1)
- 本算法主要的时间开销在于查找第i-1个元素,时间复杂度为O(n)

2.3线性表的链式表示(中)

单链表上基本操作的实现

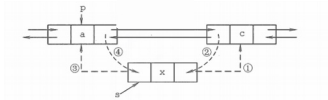
- 对某一结点进行前插操作
  - 即寻找到要插入结点的前一个结点，然后按照正常插入方法执行即可
  - 时间复杂度为O(n)
- 删除结点操作
  - 删除结点操作是将单链表的第1个结点删除。先检查删除位置的合法性,后查找表中第1-1 个结点,即被删结点的前驱结点,再将其删除
  - 示意图



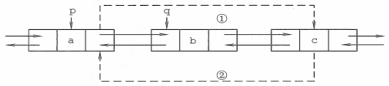
  - 该算法
- 删除结点*p
  - 方法一
    - 要删除某个给定结点*p,通常的做法是先从链表的头结点开始顺序找到其前驱结点,然后再执行删除操作
    - 算法的时间复杂度为 O(n)
  - 方法二
    - 删除结点*p的操作可用删除*p的后继结点操作来实现,实质就是将其后继结点的值赋予其自身,然后删除后继结点
    - 时间复杂度为O(1)。
- 求表长操作
  - 从第一个结点开始顺序依次访问表中的每个结点，为此需要设置一个计数器变量，每访问一个结点，计数器加1,直到访问到空结点为止
  - 算法的时间复杂度为 O(n)

双链表

- 单链表的缺点
  - 单链表只能从头结点依次顺序地向后遍历
  - 不能够快速的对其前驱结点进行操作
- 概念　双链表结点中有两个指针prior和next,分 别指向其前驱结点和后继结点
- 双链表的插入操作



  - 示意图
  - 原则就是在进行双链表插入的时候要保证不会造成断链
- 双链表的删除操作



  - 示意图
  - 原则仍然是在进行操作的时候一定要考虑是否会造成断链

操作的时候一定要考虑，本操作顺序是否会造成后继结点的丢失

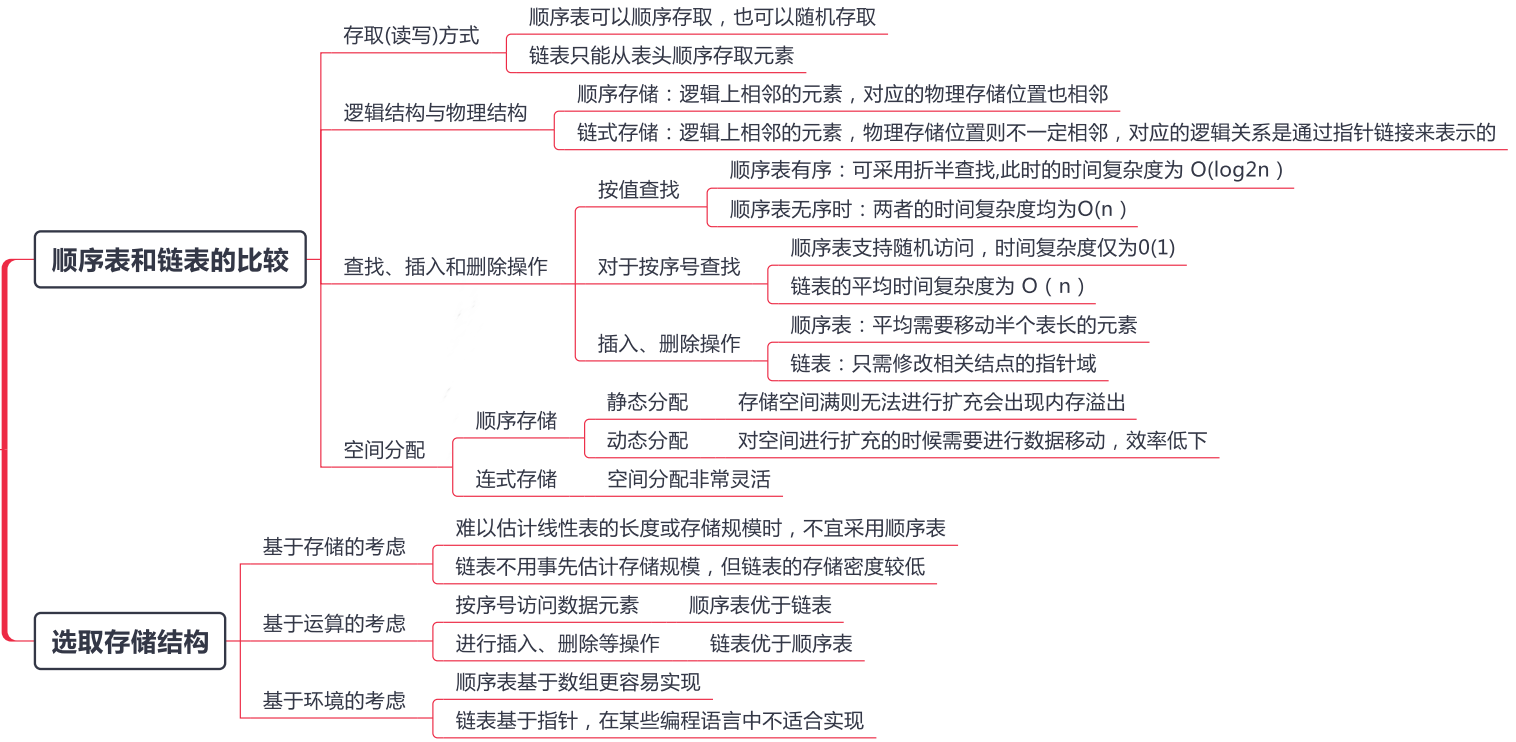
循环链表

- 循环单链表
  - 循环单链表和单链表的区别在于,表中最后一个结点的指针不是NULL,而改为指向头结点，形成了一个环
  - 判空条件：头结点的指针是否等于头指针
  - 仅设尾指针
    - 如果是要在链首之前插入结点，此时效率明显更高
    - 时间复杂度O（1）
- 循环双链表
  - 在循环双链表中,头结点的 prior指针还要指向表尾结点
  - 在循环双链表L中,某结点*p为尾结点时,p->next==L
  - 当循环双链表为空表时,其头结点的 prior 域和 next域都等于L

静态链表

- 基本结构
  - 静态链表借助数组来描述线性表的链式存储结构　静态链表也要预先分配一块连续的内存空间
  - 结点也有数据域data和指针域next　这里的指针是结点的相对地址(数组下标)，又称游标
- 特点
  - 静态链表以 next==-1作为其结束的标志
  - 静态链表的插入、删除操作与动态链表的相同, 只需要修改指针,而不需要移动元素
- 优点：增、删 操作不需要大量移动元素
- 缺点：不能随机存取，只能从头结点开始依次往后查找；容量固定不可变
- 适用场景：①不支持指针的低级语言；②数据元素数量固定不变的场景（如操作系统的文件分配表FAT）

2.3线性表的链式表示(下)



3.1 栈

栈的基本概念

只允许在一端进行插入或删除操作的线性表　先进后出

结构
- 栈顶（Top）：线性表允许进行插入删除的那一端
- 栈底（Bottom）：固定的，不允许进行插入和删除的另一端

卡特兰数　n个不同元素进栈,出栈元素不同排列的个数为 $\frac{1}{n+1}C_{2n}^{n}$

栈的基本操作
- InitStack (&S):初始化一个空栈S
- StackEmpty (S):判断一个栈是否为空，若栈S为空则返回true.否则返回falseo Push (&S, x):进栈,若栈S未满,则将x加入使之成为新栈顶
- Pop (&S， &x):出栈,若栈S非空,则弹出栈顶元素，并用x返回
- GetTop(S，&x)：读栈顶元素，若栈S非空，则用x返回栈顶元素
- DestroyStack (&S):销毁栈，并释放栈S占用的存储空间("&"表示引用调用)

在解答算法题时，若题干未做出限制，则可直接使用这些基本的操作函数

栈的顺序存储结构

顺序栈的实现

采用顺序存储的栈称为顺序栈，它利用一组地址连续的存储单元存放自栈底到栈顶的数据元素，同时附设一个指针(top)指示当前栈顶元素的位置

基本结构与操作
- 栈顶指针:S.top,初始时设置S.top=-1
- 栈顶元素:S.data[S.top]
- 进栈操作:栈不满时,栈顶指针先加1,再送值到栈顶元素
- 出栈操作:栈非空时,先取栈顶元素值,再将栈顶指针减1
- 栈空条件:S.top==-1
- 栈满条件:S.top==MaxSize-1;栈长:S.top+1

顺序栈的基本操作

初始化
```
void InitStack(SqStack &S){
   S.top=-1;                    //初始化栈顶指针
}
```

判栈空
```
bool StackEmpty(SqStack S){
   if(S.top==-1)                //栈空
      return true;
   else                         //不空
      return false;
}
```

进栈
```
bool Push(SqStack &S,ElemType x){
  if(S.top==MaxSize-1)          //栈满，报错
    return false;
  S.data[++S.top]=x;            //指针先加 1，再入栈
  return true;
}
```

出栈
```
bool Pop(SqStack &S,ElemType &x){
   if(S.top==-1)                //栈空，报错
      return false;
   x=S.data[S.top--];           //先出栈，指针再减 1
   return true;
}
```

读栈顶元素
```
bool GetTop(SqStack S,ElemType &x){
   if(S.top==-1)                //栈空，报错
      return false;
   x=S.data[S.top];             //x 记录栈顶元素
   return true;
}
```

共享栈

让两个顺序栈共享一个一维数组空间，将两个栈的栈底分别设置在共享空间的两端，两个栈顶向共享空间的中间延伸

基本原则
- 两个栈的栈顶指针都指向栈顶元素,top0=-1时0号栈为空,top1=MaxSize时1号栈为空
- 当两个栈顶指针相邻（topl-top0=1）时,判断为栈满
- 当0号栈进栈时top0先加1 再赋值,1号栈进栈时 top1先减1再赋值;出栈时则刚好相反

存取数据的时间复杂度均为O(1)

栈的链式存储结构

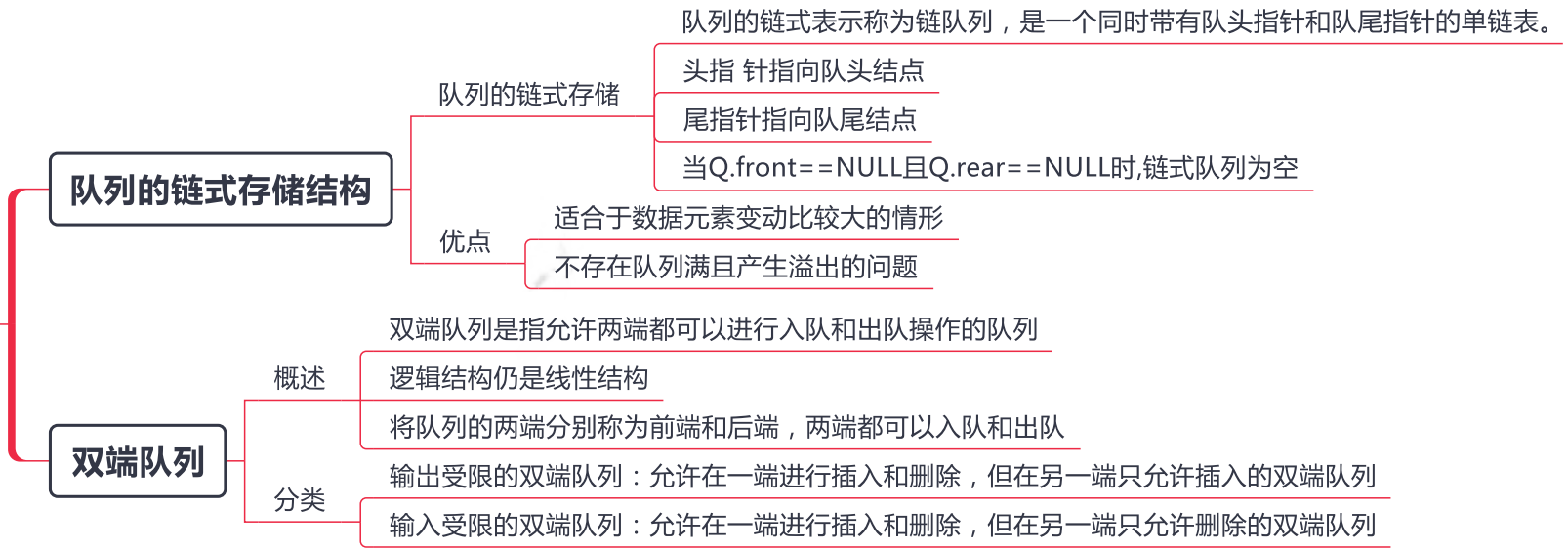
采用单链表实现，并规定所有操作都是在单链表的表头进行

优点
- 便于多个栈共享存储空间和提高其效率
- 且不存在栈满上溢的情况

3.2队列（上）

- **队列的基本概念**
  - 队列的定义
    - 是一种操作受限的线性表，只允许在表的一端进行插入，而在表 的另一端进行删除
    - 向队列中插入元素称为入队或进 队；删除元素称为出队或离队
    - 先进先出
    - 结构
      - 队头(Front）：允许删除的一端，又称队首
      - 队尾(Rear)：允许插入的一端
      - 空队列：不含任何元素的空表
  - 队列基本操作
    - InitQueue (&Q):初始化队列，构造一个空队列Q
    - QueueEmpty (Q):判队列空，若队列Q为空返回true.否则返回false
    - EnQueue (&Q，x):入队，若队列Q未满，将x加入，使之成为新的队尾
    - DeQueue (&Q，&x)：出队，若队列Q非空，删除队头元素，并用x返回
    - GetHead(Q，&x)：读队头元素，若队列Q非空，则将队头元素赋值给X
- **队列的顺序存储结构**
  - 队列的顺序存储
    - 分配一块连续的存储单元存放队列中的元素，并附设两个指针
    - 队头指针front指向队头元素，队尾指针rear指向队尾元素的下一个位置（具体问题具体分析）
    - 基本操作
      - 初始状态（队空条件）:Q.front==Q.rear==0
      - 进队操作:队不满时,先送值到队尾元素,再将队尾指针加1
      - 出队操作:队不空时,先取队头元素值,再将队头指针加1
    - 假溢出
      - 这种溢出并不是真正的溢出,在data 数组中依然存在可以存放元素的空位置
  - 循环队列
    - 把存储队列元素的表从逻辑上视为一个环，称为循环队列
    - 基本操作
      - 初始时:Q.front=Q.rear=0
      - 队首指针进1:Q.front=（Q.front+1）%MaxSize
      - 队尾指针进1:Q.rear=(Q.rear+1)%MaxSize
      - 队列长度:(Q.rear+MaxSize-Q.front)%MaxSize。
    - 判断条件
      - 队空：Q.front==Q.rear
      - 队满
        - 牺牲一个单元来区分队空和队满,即"队头指针在队尾指针的下一位置作为队满的标志"
          - 队满条件:(Q.rear+1)%MaxSize==Q.front
          - 队空条件:Q.front==Q.rear
          - 队列中元素的个数:（Q.rear-Q.front+MaxSize）%MaxSize
        - 类型中增设表示元素个数的数据成员。
          - 队空：为Q.size==0
          - 队满：Q.size==MaxSize
        - 类型中增设 tag 数据成员,以区分是队满还是队空
          - tag 等于0时,若因删除导致Q.front==Q.rear,则为队空
          - tag等于1时,若因插入导致Q.front==Q.rear,则为队满

3.2队列（下）


队列的链式存储结构
队列的链式存储
队列的链式表示称为链队列，是一个同时带有队头指针和队尾指针的单链表。
头指 针指向队头结点
尾指针指向队尾结点
当Q.front==NULL且Q.rear==NULL时,链式队列为空
优点
适合于数据元素变动比较大的情形
不存在队列满且产生溢出的问题
双端队列
概述
双端队列是指允许两端都可以进行入队和出队操作的队列
逻辑结构仍是线性结构
将队列的两端分别称为前端和后端，两端都可以入队和出队
分类
输出受限的双端队列：允许在一端进行插入和删除，但在另一端只允许插入的双端队列
输入受限的双端队列：允许在一端进行插入和删除，但在另一端只允许删除的双端队列

3.3~4矩阵的压缩存储（上）



3.3~4矩阵的压缩存储（下）

特殊矩阵的压缩存储

- 数组的定义
  - 概述
    - 数组是由n个相同类型的数据元素构成的有限序列
    - 每个数据元素称为一个数组元素
    - 每个元素在n个线性关系中的序号称为该元素的下标,下标的取值范围称为数组的维界　下标从0开始计数
  - 数组与线性表的关系
    - 数组是线性表的推广
    - 一维数组可视为一个线性表；二维数组可视为 其元素也是定长线性表的线性表
  - 特点　数组一旦被定义，其维数和维界就不再改变
- 数组的存储结构
  - 一个数组的所有元素在内存中占用一段连续的存储空间
  - 多维数组　映射方法
    - 按行优先
      - 按照一行一行的存储
      - 二维数组公式 $LOC(a_{i,j}) = LOC(a_{0,0}) + [i \times (h_2 + 1) + j] \times L$
    - 按列优先
      - 按照一列一列的存储
      - 二维数组公式 $LOC(a_{i,j}) = LOC(a_{0,0}) + [j \times (h_1 + 1) + i] \times L$

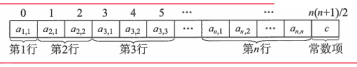
考试的时候可以根据具体问题现场推演

矩阵的压缩存储

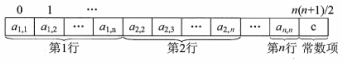
- 概述
  - 压缩存储
    - 指为多个值相同的元素只分配一个存储空间，对零元素不分配存储空间
    - 目的是为了节省存储空间
  - 特殊矩阵
    - 指具有许多相同矩阵元素或零元素，并且这些相同矩阵元素或零元素的分布有一 定规律性的矩阵
    - 常见的特殊矩阵
      - 对称矩阵
      - 上（下）三角矩阵
      - 对角矩阵
  - 特殊矩阵的压缩存储方法：找出特殊矩阵中值相同的矩阵元素的分布规律，把那些呈现规律 性分布的、值相同的多个矩阵元素压缩存储到一个存储空间中
- 特殊矩阵
  - 对称矩阵
    - 概念
      - 关于左对角线对称的矩阵
      - 上三角区：位于对角线上方的区域
      - 下三角区：位于对角线下方的区域
    - 计算公式 $k = \begin{cases} \frac{i(i-1)}{2} + j - 1, & i \geq j \text{（下三角区和主对角线元素）} \\ \frac{j(j-1)}{2} + i - 1, & i < j \text{（上三角区元素} a_{ij} = a_{ji} \text{）} \end{cases}$

记不住公式的话记住推导过程，考试的时候可以直接带入试试

  - 三角矩阵
    - 下三角矩阵
      - 上三角区的所有元素均为同一常量
      - 存储完下三角区和主对角线上的元素之后,紧接着存储对角线上方的常量一次
      - 元素下标之间的对应关系 $k = \begin{cases} \frac{i(i-1)}{2} + j - 1, & i \geq j \text{（下三角区和主对角线元素）} \\ \frac{n(n+1)}{2}, & i < j \text{（上三角区元素）} \end{cases}$
      - 下三角矩阵在内存中的压缩存储形式



    - 上三角矩阵
      - 下三角区的所有元素均为同一常量
      - 只需存储主对角线、上三角区上的元素和下三角区的常量一次
      - 元素下标之间的对应关系 $k = \begin{cases} \frac{(i-1)(2n-i+2)}{2} + (j-i), & i \leq j \text{（上三角区和主对角线元素）} \\ \frac{n(n+1)}{2}, & i > j \text{（下三角区元素）} \end{cases}$
      - 上三角矩阵在内存中的压缩存储形式



  - 三对角矩阵
    - 又称为带状矩阵，以角线为中心的3条对角线的区域，其他区域的元素都为零
    - 三对角矩阵A也可以采用压缩存储,将3条对角线上的元素按行优先方式存放在一维数组B中,且a,存放于B[0]中
    - 在一维数组B中存放的下标为k=2i＋j-3
  - 稀疏矩阵
    - 矩阵中非零元素的个数t,相对矩阵元素的个数s来说非常少,即s>>t的矩阵称为稀疏矩阵
    - 存储方式　三元组　将非零元素及其相应的行和列构成一个三元组(行标，列标，值)
    - 注意：稀疏矩阵压缩存储后便失去了随机存取特性
    - 稀疏矩阵的三元组既可以采用数组存储,也可以采用十字链表法存储

4.1串的定义和实现

- **基本概述**
  - 串是由零个或多个字符组成的有限序列
  - 串中任意个连续的字符组成的子序列称为该串的子串，包含子串的串相应地称为主串
  - 子串在主串中的位置以子串的第一个字符在主串中 的位置来表示
  - 当两个串的长度相等且每个对应位置的字符都相等时，称这两个串是相等的
  - 由一个或多个空格（空格是特殊字符）组成的串称为空格串　　其长度为串中空格字符的个数
- **串的存储结构**
  - 定长顺序存储表示
    - 类似于线性表的顺序存储结构，用一组地址连续的存储单元存储串值的字符序列
    - 截断：串的实际长度只能小于等于MAXLEN,超过预定义长度的串值会被舍去
    - 串表示方法
      - 1.用一个额外的变量来存放串的长度
      - 2.串值后面加一个不计入串长的结束标记字符"\0",此时的串长为隐含值
    - 弊端
      - 串值序列的长度超过上界,约定用截断法处理
      - 解决方法　　用不限定串长的最大长度，即采用动态分配的方式
  - 堆分配存储表示　　堆分配存储表示仍然以一组地址连续的存储单元存放串值的字符序列，但它们的存储空间是在程序执行过程中动态分配得到的
  - 块链存储表示
    - 每个结点既可以存放一个字符,也可以存放多个字符。每个结点称为块,整个链表称为块链结构
    - 示意图



- **串的基本操作**
  - StrAssign（T,chars）:赋值操作。把串T赋值为chars
  - StrCopy(&T,S）:复制操作。由串S 复制得到串T
  - StrEmpty（(S）:判空操作。若S为空串,则返回TRUE,否则返回FALSE
  - Strcompare（(S,T）:比较操作。若S>T,则返回值>0;若 s=T,则返回值=0;若s<T, 则返回值<0
  - StrLength(S):求串长。返回串S的元素个数
  - SubString(&Sub,S,pos,len）:求子串。用Sub返回串S的第pos个字符起长度为len的子串
  - Concat(&T,S1,S2）:串联接。用T返回由S1和S2联接而成的新串
  - Index（S,T）:定位操作。若主串S中存在与串T值相同的子串,则返回它在主串S中第一次出现的位置;否则函数值为0
  - clearString(&S）:清空操作。将S清为空串
  - DestroyString(&S）:销毁串。将串S销毁
- **字符集编码**
  - 英文字符——ASCII字符集
  - 中英文——Unicode字符集

4.2串的模式匹配

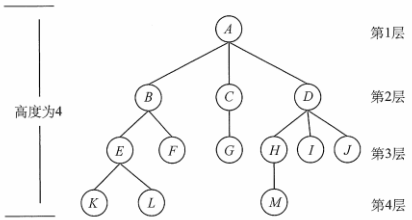
- **简单的模式匹配算法**
  - 子串的定位操作通常称为串的模式匹配，它求的是子串(常称模式串)在主串中的位置
  - 实现思想
    - 将主串中与模式串长度相同的子串搞出来，挨个与模式串对比
    - 当子串与模式串某个对应字符不匹配时，就立即放弃当前子串，转而检索下一个子串
  - 缺点：主串指针会出现回溯现象导致时间开销增加
  - 最坏时间复杂度：O(mm），其中n和m分别为主串和模式串的长度。
- **改进的模式匹配算法—KMP算法**
  - 字符串的前缀、后缀和部分匹配值
  - next 数组：当模式串的第 j 个字符匹配失败时，令模式串跳到 next[j] 再继续匹配
  - 获取串的匹配值　'ababa'为例
    - 'a'的前缀和后缀都为空集,最长相等前后缀长度为0
    - 'ab'的前缀为{a),后缀为(b},(a}n（b}=O,最长相等前后缀长度为0
    - 'aba'的前缀为{a,ab},后缀为{a,ba),（a,ab）n{a,ba）={a），最长相等前后缀长度为1
    - 'abab'的前缀{a,ab,aba）n后缀{b,ab,bab}=(ab），最长相等前后缀长度为2
    - 'ababa'的前缀{a,ab,aba,abab)门后缀{a,ba,aba,baba}={a,aba},公共元素有两个,最长相等前后缀长度为3
    - 故字符串'ababa'的部分匹配值为00123
  - 当子串和模式串不匹配时，主串指针 i 不回溯，模式串指针 j=next[j]
  - 时间复杂度：O（m+n）
  - 优点：主串不会进行回溯
- **KMP算法的进一步优化**
  - KMP算法优化：当子串和模式串不匹配时j=nextval[j]　通过构造nextval函数

5.1树的基本概念

树的定义

- 树是n（n之0）个节点的有限集。当n=0时,称为空树
- 在任意一棵非空树中应满足
  - 有且仅有一个特定的称为根的结点
  - 当n>1时,其余节点可分为m（m>0）个互不相交的有限集T1,T2....Tm,其中每个集合本身又是一棵树,并且称为根的子树
- 树是一种逻辑结构，也是一种分层结构
  - 树的根结点没有前驱,除根结点外的所有结点有且只有一个前驱
  - 树中所有结点可以有零个或多个后继

基本术语



示意图

- 考虑结点K
  - K的祖先：根A到结点K的唯一路径上的任意结点
  - 子孙：结点B是结点K的祖先,而结点K是结点B的子孙
  - 双亲与孩子：路径上最接近结点K的结点E称为K的双亲, 而K为结点E的孩子
  - 兄弟：有相同双亲的结点称为兄弟,如结点K和结点L有相同的双亲E,即K和L为兄弟
- 结点的度：树中一个结点的孩子个数，树中结点的最大度数称为树的度
- 分支结点：度大于0的结点
- 叶子结点（又称终端结点）：度为0（没有子女结点）的结点
- 结点的层次：从树根开始定义，根结点为第1层，它的子结点为第2层，以此类推
- 结点的深度：从根结点开始自顶向下逐层累加的
- 结点的高度：从叶结点开始自底向上逐层累加的
- 树的高度（或深度）：树中结点的最大层数
- 有序树和无序树：树中结点的各子树从左到右是有次序的，不能互换，称该树为有序树，否则称为无序树
- 路径和路径长度：树中两个结点之间的路径是由这两个结点之间所经过的结点序列构成的，而路径长度是路径上所经过的边的个数
- 森林：森林是n棵互不相交的树的集合
  - 只要把树的根结点删去就成了森林
  - 给农棵独立的树加上一个结点，并把这m棵树作为该结点的子树，则森林就变成了树

树的性质

- 树中的结点数等于所有结点的度数加1
- 度为m的树中第i层上至多有 m^{i-1} 个结点（i大于等于1）
- 高度为h的m叉树至少有 h 个结点。
- 高度为h的m叉树至多有 $(m^h-1)/(m-1)$ 个结点
- 高度为h、度为m的树至少有 h+m-1 个结点
- 具有n个结点的m叉树的最小高度为 $\lceil \log_m(n(m-1)+1) \rceil$
- m叉树——每个结点最多只能有m个孩子的树
  - 任意结点的度 ≤ m（最多m个孩子）
  - 允许所有结点的度都 < m
  - 可以是空树
- 度为m的树
  - 任意结点的度 ≤ m（最多m个孩子）
  - 至少有一个结点度 = m（有m个孩子）
  - 一定是非空树，至少有m+1个结点

5.2二叉树的概念

二叉树的定义及其主要特性

二叉树的定义：二叉树是另一种树形结构，其特点是每个结点至多只有两棵子树，并且二叉树的子树有左右之分，其次序不能任意颠倒

特殊的二叉树

二叉树与度为2的有序树的区别
- 度为2的树至少有3个结点，而二叉树可以为空
- 孩子结点
  - 度为2的有序树的孩子的左右次序是相对于另一孩子而言的，若某个结点只有一个孩子，则这个孩子就无须区分其左右次序
  - 二叉树无论其孩子数是否为2,均需确定其左右次 序

满二叉树
- 一棵高度为h,且含有 2^h-1 个结点的二叉树称为满二叉树
- 对于编号为i的结点
  - 双亲为 $\lfloor i/2 \rfloor$
  - 若有左孩子,则左孩子为2i
  - 若有右孩子,则右孩子为2i + 1

完全二叉树
- 高度为h、有n个结点的二叉树,当且仅当其每个结点都与高度为h的满二叉树中编号为1 ~ n的结点一一对应时,称为完全二叉树
- $i \le \lfloor n/2 \rfloor$ 则结点为分支结点，否则为叶子结点
- 特点
  - 叶子结点只可能在层次最大的两层上出现。对于最大层次中的叶子结点，都依次排列 在该层最左边的位置上
  - 若有度为1的结点，则只可能有一个，且该结点只有左孩子而无右孩子（重要特征）
  - 按层序编号后,一旦出现某结点（编号为i）为叶子结点或只有左孩子,则编号大于i 的结点均为叶子结点
  - 若n为奇数,则每个分支结点都有左孩子和右孩子
  - 若n为偶数,则编号最大的分支结点（编号为n/2）只有左孩子,没有右孩子,其余分支结点左、右孩子都有

二叉排序树 左子树上所有结点的关键字均小于根结点的关键字;右子树上的所有结点的关键字均大于根结点的关键字;左子树和右子树又各是一棵二叉排序树

平衡二叉树：树上任一结点的左子树和右子树的深度之差不超过1

二叉树的性质
- 非空二叉树上的叶子结点数等于度为2的结点数加1
- 非空二叉树上第k层上至多 2^{k-1} 个结点
- 高度为h的二叉树至多有 2^h-1 个结点
- 对完全二叉树按从上到下、从左到右的顺序依次编号1,2,...,n,则有以下关系
- 当i>1时,结点i的双亲的编号为 $\lfloor i/2 \rfloor$ 即当i为偶数时,其双亲的编号为i/2它是双亲的左孩子;当i为奇数时,其双亲的编号为(i-1)/2,它是双亲的右孩子
- $2i \le n$ 结点i的左孩子编号为2i,否则无左孩子
- $2i+1 \le n$ 结点i的右孩子编号为2i + 1,否则无右孩子
- 结点7所在层次（深度）为 $\lfloor \log_2 i \rfloor + 1$
- 具有n个（n>0）结点的完全二叉树的高度为 $\lceil \log_2(n+1) \rceil$ 或 $\lfloor \log_2 n \rfloor + 1$

二叉树的存储结构

顺序存储结构
- 用一组地址连续的存储单元依次自上而下、自左至右存储完全二叉树 上的结点元素，即将完全二叉树上编号为i的结点元素存储在一维数组下标为i - 1的分量中
- 完全二叉树和满二叉树采用顺序存储比较合适
- 一般的二叉树，为了能反映二叉树中结点之间的逻辑关系，只能添加并不存在的空结点，让其每个结点与完全二叉树上的结点相对照，再存储到一维数组的相应分量 中

链式存储结构
- 由于顺序存储的空间利用率较低，因此二叉树一般都采用链式存储结构，用链表结点来存储二叉树中的每个结点
- 结构描述

```
typedef struct BiTNode{
    ElemType data;                  //数据域
    struct BiTNode  *lchild,*rchild;    //左、右孩子指针
}BiTNode,*BiTree;
```

- 在含有n个结点的二叉链表中,含有n + 1个空链域

5.3二叉树的遍历和线索二叉树

- 二叉树的遍历
  - 先序遍历
    - 访问根结点
    - 先序遍历左子树
    - 先序遍历右子树
  - 中序遍历
    - 中序遍历左子树
    - 访问根结点
    - 中序遍历右子树
  - 后序遍历
    - 后序遍历左子树
    - 后序遍历右子树
    - 访问根结点
  - ***时间复杂度o(n)***
    - 递归算法和非递归算法的转换：利用栈进行实现
- 层次遍历
  - 利用队列实现
  - 实现
    - 先将二叉树根结点入队，然 后出队，访问出队结点
    - 若它有左子树，则将左子树根结点入队
    - 若它有右子树，则将右子树根结点入队
    - 然后出队，访问出队结点……如此反复，直至队列为空
- 由遍历序列构造二叉树
  - 二叉树的先序（后序）序列和中序序列可以唯一地确定一棵二叉树
  - 层序遍历与中序（后序）可以确定一颗二叉树
- 线索二叉树
  - 基本概念
    - 线索二叉树的基本概念
      - 遍历二叉树是以一定的规则将二叉树中的结点排列成一个线性序列，从而得到几种遍历序列
      - 使得该序列中的每个结点（第一个和最后一个结点除外）都有一个直接前驱和直接后继
    - 规定
      - 若无左子树，令lchild指向其前驱结点
      - 若无右子树，令rchild指向其后继结点
    - 结点结构

lchild	ltag	data	rtag	rchild
    - 标志域的含义
      - ltag
        - 0 lchild域指示结点的左孩子
        - 1 lchild域指示结点的前驱
      - rtag
        - 0 rchild域指示结点的右孩子
        - 1 rchild域指示结点的后继
  - 中序线索二叉树的构造
    - 概念
      - 二叉树的线索化是将二叉链表中的空指针改为指向前驱或后继的线索
      - 而前驱或后继的信息只有在遍历时才能得到,因此线索化的实质就是遍历一次二叉树
    - 中序线索二叉树的建立
      - 附设指针 pre指向刚刚访问过的结点,指针p指向正在访问的结点,即pre指向p的前驱
      - 在中序遍历的过程中
        - 检查p的左指针是否为空若为空就将它指向pre
        - 检查pre的右指针是否为空,若为空就将它指向p
    - 中序线索二叉树的遍历
      - 若其右标志为"1",则右链为线索,指示其后继,否则遍历右子树中第一个访问的结点（右子树中最左下的结点）为其后继
  - 先序线索二叉树
    - 先序序列为ABCDF,然后依次判断每个结点的左右链域,如果为空则将其改造为线索
    - 结点A,,B均有左右孩子;结点C无左孩子,将左链域指向前驱B,无右孩子,将右链域指向后继D
    - 结点D无左孩子,将左链域指向前驱C,无右孩子,将右链域指向后继F
    - 结点F无左孩子,将左链域指向前驱D,无右孩子, 也无后继故置空
  - 后序线索二叉树
    - 后序序列为CDBFA,结点C无左孩子,也无前驱故置空,无右孩子,将右链域指向后继D
    - 结点D无左孩子,将左链域指向前驱C,无右孩子,将右链域指向后继B
    - 结点F无左孩子,将左链域指向前驱B,无右孩子,将右链域指向后继A

5.4树、森林

树的存储结构

双亲表示法

采用一组连续空间来存储每个结点，同时在每个结点中增设一个伪指针，指示 其双亲结点在数组中的位置

根结点下标为0,其伪指针域为-1

优点　利用了每个结点（根结点除外）只有唯一双亲的性质,可以很快得到每个结点的双亲结点

缺点　但求结点的孩子时需要遍历整个结构

孩子表示法

将每个结点的孩子结点都用单链表链接起来形成一个线性结构,此时n个结点就有n个孩子链表（叶子结点的孩子链表为空表）

优点　寻找子女的操作非常直接

缺点　寻找双亲的操作需要遍历n个结点中孩子链表指针域所指向的n个孩子链表

孩子兄弟表示法

以二叉链表作为树的存储结构，孩子兄弟表示法使每个结点包括三部分内容:结点值、指向结点第一个孩子结点的指针,及指向结点下一个兄弟结点的指针

优点　这种存储表示法比较灵活,其最大的优点是可以方便地实现树转换为二叉树的操作,易于查找结点的孩子等

缺点　从当前结点查找其双亲结点比较麻烦

若为每个结点增设一个parent 域指向其父结点,则查找结点的父结点也很方便

树、森林与二叉树的转换

树转换为二叉树的规则

每个结点左指针指向它的第一个孩子，右指针指向它在树中的相邻右兄弟

树转换成二叉树的画法

在兄弟结点之间加一连线

对每个结点，只保留它与第一个 孩子的连线，而与其他孩子的连线全部抹掉

以树根为轴心，顺时针旋转45度



森林转换成二叉树的画法

将森林中的每棵树转换成相应的二叉树

每棵树的根也可视为兄弟关系,在每棵树的根之间加一根连线

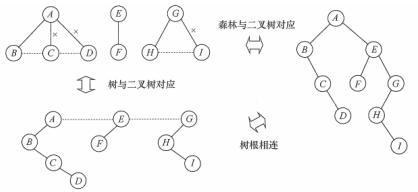
以第一棵树的根为轴心顺时针旋转45°

二叉树转换为森林的规则

若二叉树非空,则二叉树的根及其左子树为第一棵树的二叉树形式,故将根的右链断开

二叉树根的右子树又可视为一个由除第一棵树外的森林转换后的二叉树

应用同样的方法,直到最后只剩一棵没有右子树的二叉树为止,最后再将每棵二叉树依次转换成树,就得到了原森林



森林与二叉树的对应关系

树和森林的遍历

先根遍历

若树非空,先访问根结点,再依次遍历根结点的每棵子树,遍历子树时仍遵循先根后子树的规则

其遍历序列与这棵树相应二叉树的先序序列相同

后根遍历

若树非空,先依次遍历根结点的每棵子树,再访问根结点,遍历子树时仍遵循先子树后根的规则

其遍历序列与这棵树相应二叉树的中序序列相同

先序遍历森林

访问森林中第一棵树的根结点

先序遍历第一棵树中根结点的子树森林

先序遍历除去第一棵树之后剩余的树构成的森林

中序遍历森林

中序遍历森林中第一棵树的根结点的子树森林

访问第一棵树的根结点

中序遍历除去第一棵树之后剩余的树构成的森林

树和森林的遍历与二叉树遍历的对应关系

树	森林	二叉树
先根遍历	先序遍历	先序遍历
后根遍历	中序遍历	中序遍历

树的应用——并查集

支持操作

1）Union（S,Root1,Root2）:把集合S中的子集合Root2并入子集合Root1。要求Root1 和Root2互不相交,否则不执行合并

2）Find(S,x):查找集合S中单元素x所在的子集合,并返回该子集合的名字

3）Initial(S）:将集合S中的每个元素都初始化为只有一个单元素的子集合

具体实现

用树（森林）的双亲表示作为并查集的存储结构,每个子集合以一棵树表示

所有表示子集合的树,构成表示全集合的森林,存放在双亲表示数组内

通常用数组元素的下标代表元素名,用根结点的下标代表子集合名,根结点的双亲结点为负数

5.5树与二叉树的应用（上）

二叉排序树（BST）

- 二叉排序树的定义
  - 特性
    - 1）若左子树非空,则左子树上所有结点的值均小于根结点的值
    - 2）若右子树非空,则右子树上所有结点的值均大于根结点的值
    - 3）左、右子树也分别是一棵二叉排序树

  左子树结点值<根结点值< 右子树结点值
- 二叉排序树的查找
  - 二叉排序树的查找是从根结点开始,沿某个分支逐层向下比较的过程，若二叉排序树非空,先将给定值与根结点的关键字比较
    - 若相等,则查找成功
    - 若不等,如果小于根结点的关键字, 则在根结点的左子树上查找
    - 否则在根结点的右子树上查找

  递归
- 二叉排序树的插入
  - 插入结点的过程
    - 若原二叉排序树为空，则直接插入结点
    - 若关键字k小于根结点值，则插入到左子树
    - 若关键字k大于根结点值，则插入到右子树

  插入的结点一定是一个新添加的叶结点，且是查找失败时的查找路径上访问的最后一个结点的左孩子或右孩子
- 二叉排序树的构造
  - 设查找的关键字序列为{45,24,53,45,12,24},则生成的二叉排序树
  - 生成示意图



- 二叉排序树的删除
  - 若被删除结点z是叶结点,则直接删除,不会破坏二叉排序树的性质
  - 若结点z只有一棵左子树或右子树,则让z的子树成为z父结点的子树,替代z的位置
  - 若结点z有左、右两棵子树,则令z的直接后继（或直接前驱）替代z,然后从二叉排序树中删去这个直接后继（或直接前驱),这样就转换成了第一或第二种情况
- 二叉排序树的查找效率分析
  - 平均执行时间为

5.5树与二叉树的应用（中）

平衡二叉树

平衡二叉树的定义

在插入和删除二叉树结点时，要保证任意结点的左、右子树高度差的绝对值不超过 1,将这样的二叉树称为平衡二叉树

平衡因子：左子树与右子树的高度差为该结点的平衡因子　　平衡因子的值只可能是-1、0或1

平衡二叉树的插入

当在二叉排序树中插入（或删除）一个结点时

首先检查其插入路径上的结点是否因为此次操作而导致了不平衡

若导致了不平衡,则先找到插入路径上离插入结点最近的平衡因子的绝对值大于1的结点A

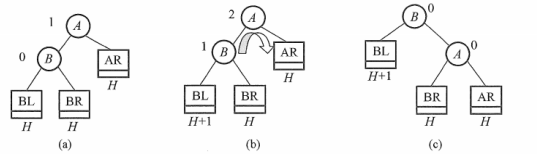
,再对以A为根的子树,在保持二叉排序树特性的前提下,调整各结点的位置关系,使之重新达到平衡

规律归纳

LL平衡旋转（右单旋转）

由于在结点A的左孩子（L）的左子树（L）上插入了新结点

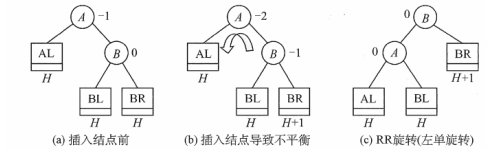
A的平衡因子由1增至2,导致以A为根的子树失去平衡,需要一次向右的旋转操作



RR平衡旋转（左单旋转）

由于在结点A的右孩子（R）的右子树（R)上插入了新结点

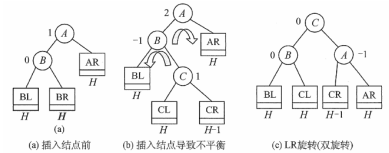
A的平衡因子由-1减至-2,导致以A为根的子树失去平衡,需要一次向左的旋转操作



LR平衡旋转（先左后右双旋转）

由于在A的左孩子（L的右子树（R）上插入新结点

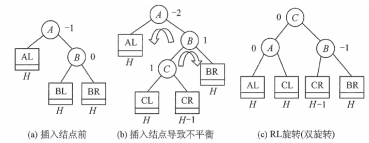
A的平衡因子由1增至2,导致以A为根的子树失去平衡,需要进行两次旋转操作,先左旋转后右旋转



RL平衡旋转（先右后左双旋转）

由于在A的右孩子（R)的左子树（L）上插入新结点

A的平衡因子由-1减至-2,导致以A为根的子树失去平衡,需要进行两次旋转操作,先右旋转后左旋转



平衡二叉树的查找

平均查找长度

5.5树与二叉树的应用（下）

哈夫曼树和哈夫曼编码

哈夫曼树的定义

称为哈夫曼树, 也称最优二叉树：在含有n个带权叶结点的二叉树中,其中带权路径长度（WPL)最小的二叉树

基本概念

结点的带权路径长度：从树的根到任意结点的路径长度(经过的边数)与该结点上权值的乘积

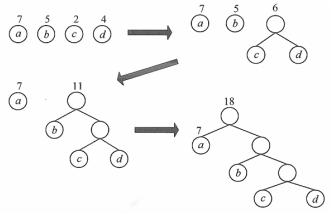
$$WPL=\sum_{i=1}^{n}w_i l_i$$

树中所有叶结点的带权路径长度之和称为该树的带权路径长度

- w_i是第i个叶结点所带的权值
- l_i是该叶结点到根结点的路径长度

哈夫曼树的构造

每次从合成后存在的结点中选出两个最小的进行构造二叉树



示意图

哈夫曼编码

基本概念

可变长度编码：在数据通信中，若对每个字符用相等长度的二进制位表示

可变长度编码：允许对不同字符用不等长的二进制位表示

特点：
- 对频率高的字符赋以短编码，而对频率较低的字符则赋以较长一些的编码
- 可以使字符的平均编码长度减短，起到压缩数据的效果

前缀编码：没有一个编码是另一个编码的前缀

可变长度编码比固定长度编码要好

哈夫曼编码是一种被广泛应用而且非常有效的数据压缩编码

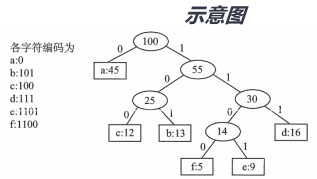
构造哈夫曼编码

将每个出现的字符当作一个独立的结点, 其权值为它出现的频度（或次数）,构造出对应的哈夫曼树

将字符的编码解释为从根至该字符的路径上边标记的序列

其中边标记为0表示"转向左孩子",标记为1表示"转向右孩子"

示意图



6.1图的基本概念

图的定义

图G由顶点集V和边集E组成,记为G=(V,E），其中 V(G)表示图G中顶点的有限非空集;E（G）表示图G中顶点之间的关系（边）集合

$V=\{v_1, v_2, \cdots, v_n\}$ 则用|V|表示图G中顶点的个数也称图G的阶　　$E=\{(u,v) \mid u \in V, v \in V\}$ 用|E|表示图G中边的条数

注意:线性表可以是空表,树可以是空树,但图不可以是空图

图的一些基本概念及术语

- 有向图
  - 若E是有向边（也称弧)的有限集合时,则图G为有向图。
  - 弧是顶点的有序对,记为<v,w>, 其中v,w是顶点,v称为弧尾,w称为弧头,<v,w>称为从顶点v到顶点w的弧,也称v邻接到w, 或 w邻接自v
- 无向图
  - 若E是无向边（简称边）的有限集合时,则图G为无向图
  - 边是顶点的无序对,记为(v,,w)或(w,v），因为(v,w）=(w,v），其中v,w是顶点
  - 可以说顶点w和顶点v互为邻接点。边(v,w）依附于顶点w和v,或者说边(v,w)和顶点y,w相关联
- 简单图
  - 不存在重复边
  - 不存在顶点到自身的边，则称图G为简单图
- 多重图　若图G中某两个结点之间的边数多于一条，又允许顶点通过同一条边和自己关联，则G为多重图
- 完全图(也称简单完全图)
  - 对于无向图,|E|的取值范围是0到n(n-1)/2,有n（n-1)2条边的无向图称为完全图,在完全图中任意两个顶点之间都存在边
  - 对于有向图,|E|的取值范围是0到n(n-1），有n(n-1)条弧的有向图称为有向完全图,在有向完全图中任意两个顶点之间都存在方向相反的两条弧
- 子图　设有两个图G=(V,E）和G'=(V',E'），若V'是V的子集,且E'是E的子集,则称G'是G的子图。若有满足V(G')=V(G)的子图G',则称其为G的生成子图
- 连通、连通图和连通分量
  - 在无向图中,若从顶点v到顶点w有路径存在,则称v和w是连通的
  - 若图G中任意两个顶点都是连通的,则称图G为连通图,否则称为非连通图
  - 无向图中的极大连通子图称为连通分量
  - 若一个图有n个顶点,并且边数小于n-1,则此图必是非连通图
- 强连通图、强连通分量
  - 强连通图：在有向图中,若从顶点v到顶点w和从顶点w到顶点v之间都有路径,则称这两个顶点是强连通的若图中任何一对顶点都是强连通的,则称此图为强连通
  - 有向图中的极大强连通子图称为有向图的强连通分量
- 生成树、生成森林
  - 连通图的生成树是包含图中全部顶点的一个极小连通子图
  - 若图中顶点数为n,则它的生成树含有n-1条边
  - 对生成树而言,若砍去它的一条边,则会变成非连通图,若加上一条边则会形成一个回路
  - 在非连通图中,连通分量的生成树构成了非连通图的生成森林
- 顶点的度，入度和出度
  - 度：定义为以该顶点为一个端点的边的数目
  - 无向图：顶点v的度是指依附于该顶点的边的条数　无向图的全部顶点的度的和等于边数的2倍
  - 有向图：全部顶点的入度之和与出度之和相等,并且等于边数
- 边的权和网　每条边都可以标上具有某种含义的数值,该数值称为该边的权值。这种边上带有权值的图称为带权图,也称网
- 稠密图、稀疏图
  - 边数很少的图称为稀疏图,反之称为稠密图
  - 一般当图G满足 $|E|<|V|\log|V|$ 可以将G视为稀疏图
- 路径、路径长度和回路
  - 路径：两个顶点之间相连的边
  - 路径长度:路径上边的数目称
  - 回路或环：第一个顶点和最后一个顶点相同的路径
  - 若一个图有n个顶点,并且有大于n-1条边,则此图一定有环
- 简单路径、简单回路
  - 简单路径：顶点不重复出现的路径
  - 简单回路：除第一个顶点和最后一个顶点外,其余顶点不重复出现的回路
- 距离
  - 从顶点u出发到顶点v的最短路径若存在,则此路径的长度称为从u到v的距离
  - 若从u到v根本不存在路径,则记该距离为无穷
- 有向树　一个顶点的入度为0、其余顶点的入度均为1的有向图,称为有向树

6.2图的存储及基本操作

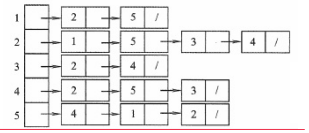
邻接矩阵法

- 邻接矩阵存储
  - 是指用一个一维数组存储图中顶点的信息,用一个二维数组存储图中边的信息（即各顶点之间的邻接关系）
  - 存储顶点之间邻接关系的二维数组称为邻接矩阵
  - 结构示意
    $$A[i][j]=\begin{cases}1, & \text{若}(v_i,v_j)\text{或}\langle v_i,v_j\rangle\text{是}E(G)\text{中的边}\\0, & \text{若}(v_i,v_j)\text{或}\langle v_i,v_j\rangle\text{不是}E(G)\text{中的边}\end{cases}$$
    - 带权图示意
      $$A[i][j]=\begin{cases}w_{ij}, & \text{若}(v_i,v_j)\text{或}\langle v_i,v_j\rangle\text{是}E(G)\text{中的边}\\0\text{或}\infty, & \text{若}(v_i,v_j)\text{或}\langle v_i,v_j\rangle\text{不是}E(G)\text{中的边}\end{cases}$$
  - 邻接矩阵表示法的空间复杂度为O(n²),其中n为图的顶点数|V|
- 特点
  - 无向图的邻接矩阵一定是一个对称矩阵（并且唯一）。因此,在实际存储邻接矩阵时只需存储上（或下）三角矩阵的元素
  - 对于无向图,邻接矩阵的第i行（或第i列）非零元素(或非0元素）的个数正好是第i 个顶点的度 $\mathrm{TD}(v_i)$
  - 对于有向图,邻接矩阵的第i行（或第i列）非零元素（或非o元素）的个数正好是第i 个顶点的出度 $\mathrm{OD}(v_i)$
  - 用邻接矩阵法存储图,很容易确定图中任意两个顶点之间是否有边相连。但是,要确定图中有多少条边,则必须按行、按列对每个元素进行检测,所花费的时间代价很大
  - 稠密图适合使用邻接矩阵的存储表示

邻接表法

当一个图为稀疏图时，使用邻接矩阵法要浪费大量的存储空间，而图的邻接表法结合了顺序存储和链式存储方法，减少了不必要的浪费

- 结构
  - 对图G中的每个顶点建立一个单链表,第i个单链表中的结点表示依附于顶点 vi的边,这个单链表就称为顶点vi的边表（对于有向图则称为出边表）
  - 边表的头指针和顶点的数据信息采用顺序存储（称为顶点表）,所以在邻接表中存在两种结点:顶点表结点和边表结点
- 示意图

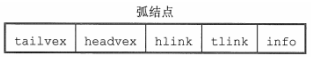


- 特点
  - 若G为无向图,则所需的存储空间为O(|V| + 2|E|);若G为有向图,则所需的存储空间为O|V|+|E|）
  - 对于稀疏图,采用邻接表表示将极大地节省存储空间
  - 在邻接表中,给定一顶点,能很容易地找出它的所有邻边,因为只需要读取它的邻接表
  - 在有向图的邻接表表示中,求一个给定顶点的出度只需计算其邻接表中的结点个数
  - 但求其顶点的入度则需要遍历全部的邻接表
  - 图的邻接表表示并不唯一

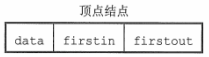
十字链表

十字链表是有向图的一种链式存储结构。在十字链表中,对应于有向图中的每条弧有一个结点,对应于每个顶点也有一个结点

- 时间复杂度O(|V| + |E|)
- 弧结点域结构
  - 尾域（tailvex）弧尾
  - 头域（headvex）：弧头
  - 链域（hlink）：指向弧头相同的下一条弧
  - 链域（tlink）：指向弧尾相同的下一条弧
  - info 域：指向该弧的相关信息



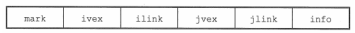
- 顶点结点域结构
  - data域存放顶点相关的数据信息
  - firstin和firstout 两个域分别指向以该顶点为弧头或弧尾的第一个弧结点



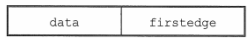
- 只能存有向图

邻接多重表

- 邻接多重表是无向图的另一种链式存储结构
- 时间复杂度O(|V| + |E|)
- 边结构
  - mark为标志域,可用以标记该条边是否被搜索过
  - ivex和jvex为该边依附的两个顶点在图中的位置
  - ilink指向下一条依附于顶点ivex的边;
  - jlink指向下一条依附于顶点jyvex 的边
  - info为指向和边相关的各种信息的指针域



- 顶点结构
  - data域存储该顶点的相关信息
  - firstedge域指示第一条依附于该顶点的边



- 只能存无向图

6.3图的遍历


广度优先搜索
广度优先搜索(BFS)
类似于二叉树的层序遍历算法
算法思想
首先访问起始顶点v,接着由v出发,依次访问v的各个未访问过的邻接顶点w1,w2...,wi
然后依次访问w1,w2,,wi的所有未被访问过的邻接顶点
再从这些访问过的顶点出发,访问它们所有未被访问过的邻接顶点,直至图中所有顶点都被访问过为止
利用队列实现搜索
BFS算法的性能分析
在最坏的情况下,空间复杂度为O(|V|)
时间复杂度
邻接表存储方式： 顶点：O(|V|) 边：O(|E|) 总时间复杂度：O(|V|+|E|)
邻接矩阵存储方式: 总时间复杂度:
BFS算法可以解决最短路径问题
广度优先生成树
在广度遍历的过程中，我们可以得到一棵遍历树，称为广度优先生成树
唯一性
邻接矩阵存储表示是唯一的，故其广度优先生成树也是唯一的
由于邻接表存储表示不唯一，故其广度优先生成树也是不唯一的
深度优先搜索
深度优先搜索（DFS）
类似于树的先序遍历。搜索策略是尽可能"深"地搜索一个图
算法思想
首先访问图中某一起始顶点v,然后由v出发,访问与v邻接且未被访问的任一顶点w1,再访问与w1邻接且未被访问的任一顶点 w2...重复上述过程。
当不能再继续向下访问时,依次退回到最近被访问的顶点,若它还有邻接顶点未被访问过,则从该点开始继续上述搜索过程,直至图中所有顶点均被访问过为止
递归算法
需要借助一个递归工作栈
DFS算法的性能分析
空间复杂度为O(|V|)
时间复杂度
邻接矩阵 时间复杂度
邻接链表
找所有顶点的邻接点所需的时间为O(|E|)
访问顶点所需的时间为O(|V|)
总时间复杂度：o (|v|+|e|)
深度优先的生成树和生成森林
对连通图调用DFS才能产生深度优先生成树
非连通图产生的是深度优先生成森林
基于邻接表存储的深度优先生成树是不唯一的
注意
基于邻接矩阵的遍历所得到的DFS序列和BFS序列是唯一的
基于邻接表的遍历所得到的DFS序列和BFS序列是不唯一的
图的遍历与图的连通性
图的遍历算法可以用来判断图的连通性
对于无向图
若无向图是连通的，则从任一结点出发，仅需一次遍历就能够访问图中的所 有顶点
若无向图是非连通的，则从某一个顶点出发，一次遍历只能访问到该顶点所在连通分量的所有顶点，而对于图中其他连通分量的顶点，则无法通过这次遍历访问
对于有向图
若无向图是连通的，从初始点到图中的每个顶点都有路径，则能够访问到图中的所有顶点
若无向图是非连通的，不能访问到所有顶点

6.4图的应用(上)


最小生成树
一个连通图的生成树包含图的所有顶点，并且只含尽可能少的边
若砍去它的一条边，则会使生成树变成非连通图
若给它增加一条边，则会形成图中的一条回路
最小生成树：权值之和最小的那棵生成树, 则称为最小生成树
最小生成树性质
最小生成树不是唯一的，即最小生成树的树形不唯一
其对应的边的权值之和总是唯一的，而且是最小的
最小生成树的边数为顶点数减1
最小生成树算法
Prim算法
概述
始时从图中任取一顶点加入树T,此时树中只含有一个顶点
之后选择一个与当前T中顶点集合距离最近的顶点,并将该顶点和相应的边加入T,每次操作后T中的顶点数和边数都增 1
以此类推,直至图中所有的顶点都并入T,得到的T就是最小生成树。此时T中必然有n-1条边
时间复杂度 $O(|V|^2)$
适用于求解边稠蜜的图的最小生成树
Kruskal 算法
概述
初始时为只有n个顶点而无边的非连通图T={V,{}},每个顶点自成一个连通分量
然后按照边的权值由小到大的顺序,不断选取当前未被选取过且权值最小的边,
若该边依附的顶点落在T中不同的连通分量上,则将此边加入T
否则舍弃此边而选择下一条权值最小的边
以此类推,直至T中所有顶点都在一个连通分量上
时间复杂度 $O(\log|E|)$
适合于边稀疏而顶点较多的图
最短路径
Dijkstra算法求单源最短路径问题
辅助数组
dist[]:记录从源点v0到其他各顶点当前的最短路径长度,它的初态为:若从v0到vi有弧,则dist[i]为弧上的权值;否则置dist[i]为无穷大
path[]:path[i]表示从源点到顶点i之间的最短路径的前驱结点
实现过程
1）初始化:集合S初始为{0},dist[]的初始值dist[i]=arcs[0][i],i=1,2,...,n-1
2)从顶点集合V-S中选出vj,满足 dist[j]=Min{dist[i]|v_i∈V-S} vj就是当前求得的一条从v0出发的最短路径的终点,令$S=S\cup\{j\}$
3）修改从V0出发到集合V-S上任一顶点vk可达的最短路径长度:若dist[j]+arcs[j][k]<dist[k],则更新dist[k]=dist[j]+arcs[j][k]
4）重复2）~3）操作共n-1次,直到所有的顶点都包含在S中
时间复杂度 $O(|V|^2)$
不适用于边权值存在负数的情况
Floyd算法求各顶点之间最短路径问题
实现过程
初始时,对于任意两个顶点vi和vj,若它们之间存在边,则以此边上的权值作为它们之间的最短路径长度
若它们之间不存在有向边,则以无穷大作为它们之间的最短路径长度
以后逐步尝试在原路径中加入顶点k（k=0,1,2....n-1）作为中间顶点
若增加中间顶点后,得到的路径比原来的路径长度减少了,则以此新路径代替原路径
时间复杂度 $O(|V|^3)$
允许图中有带负权值的边,但不允许有包含带负权值的边组成的回路
适用于带权无向图

6.4图的应用(下)

- 有向无环图描述表达式
  - 有向无环图：若一个有向图中不存在环，则称为有向无环图，简称DAG图
  - 有向无环图是描述含有公共子式的表达式的有效工具
- 拓扑排序
  - AOV网概述
    - 若用DAG图表示一个工程,其顶点表示活动,用有向边<Vi,Vj>表示活动Vi必须先于活动进行的这样一种关系,则将这种有向图称为顶点表示活动的网络,记为AOV网
    - 活动Vi是活动Vj的直接前驱,活动Vj是活动Vi的直接后继
    - 这种前驱和后继关系具有传递性,且任何活动不能以它自己作为自己的前驱或后继
  - 拓扑排序定义
    - 每个顶点出现且只出现一次
    - 若顶点A在序列中排在顶点B的前面,则在图中不存在从顶点B到顶点A的路径
  - 拓扑排序实现方法
    - ①从 AOV 网中选择一个没有前驱的顶点并输出
    - ②从网中删除该顶点和所有以它为起点的有向边
    - ③重复①和②直到当前的AOV网为空或当前网中不存在无前驱的顶点为止.后一种情况说明有向图中必然存在环
  - 注意
    - 入度为零的顶点,即没有前驱活动的或前驱活动都已经完成的顶点,工程可以从这个顶点所代表的活动开始或继续
    - 若一个顶点有多个直接后继,则拓扑排序的结果通常不唯一
    - 若各个顶点已经排在一个线性有序的序列中,每个顶点有唯一的前驱后继关系,则拓扑排序的结果是唯一的
    - 生成AOV网的新的邻接存储矩阵,可以是三角矩阵
    - 对于一般的图来说,若其邻接矩阵是三角矩阵,则存在拓扑序列;反之则不一定成立
    - 拓扑排序、逆拓扑排序序列可能不唯一
    - 若图中有环，则不存在拓扑排序序列/逆拓扑排序序列
- 逆拓扑排序
  - 具体实现　DFS算法
  - 思想
    - ①从AOV网中选择一个没有后继（出度为0）的顶点并输出
    - ② 从网中删除该顶点和所有以它为终点的有向边
    - ③重复①和②直到当前的AOV网为空
- 关键路径
  - AOE网概述　在带权有向图中，以顶点表示事件，以有向边表示活动，以边上的权值表示完成该活动的开 销（如完成活动所需的时间），称之为用边表示活动的网络
  - AOE与AOV
    - 相同点　AOE网和AOV网都是有向无环图
    - 不同点　AOE网中的边有权值；而 AOV网中的边无权值
  - AOE网性质
    - 只有在某顶点所代表的事件发生后，从该顶点出发的各有向边所代表的活动才能开始
    - 只有在进入某顶点的各有向边所代表的活动都已结束时，该顶点所代表的事件才能发生
  - 关键路径与关键活动
    - 关键路径：从源点到汇点的所有路径中,具有最大路径长度的路径
    - 关键活动：关键路径上的活动
  - 变量含义
    - 事件vk的最早发生事件ve(k)
      - ve(源点)=0
      - ve(k)=Max{ve(j)+Weight(vj,vk)｝},vk为vj的任意后继,Weight (vj,vk)表示<vj,vk>上的权值
    - 事件vk的最迟发生事件vl(k)
      - vl(汇点)= ve（汇点)
      - vl(k)=Min{vl(j)-Weight(vk,vj)},vk为vj的任意前驱
    - 活动ai的最早开始时间 e(i)　该活动弧的起点所表示的事件的最早发生时间。若边<vk,vj>表示活动ai,则有e（i）=ve(k)
    - 活动ai的最迟开始时间l(i)　该活动弧的终点所表示事件的最迟发生时间与该活动所需时间之差。若边<vk,vj>表示活动ai,则有l(i)=vl(j)-Weight(vk, vj)
    - 一个活动ai的最迟开始时间l(i)和其最早开始时间e(i)的差额d(i)=l（i)-e(i)
  - 关键路径的算法步骤
    - 从源点出发,令ve(源点)=0,按拓扑有序求其余顶点的最早发生时间 ve()
    - 从汇点出发,令vl(汇点）=ve（汇点）,按逆拓扑有序求其余顶点的最迟发生时间vl()
    - 根据各顶点的ve()值求所有弧的最早开始时间e( )
    - 根据各顶点的 vl()值求所有弧的最迟开始时间l()
    - 求AOE 网中所有活动的差额d(),找出所有d()=0的活动构成关键路径
  - 注意
    - 关键路径上的所有活动都是关键活动,是决定整个工程的关键因素,因此可通过加快关键活动来缩短整个工程的工期
    - 不能任意缩短关键活动,因为一旦缩短到一定的程度,该关键活动就可能会变成非关键活动
    - 网中的关键路径并不唯一,只有加快那些包括在所有关键路径上的关键活动才能达到缩短工期的目的
    - 若关键活动耗时增加，则整个工程的工期将增长
    - 缩短关键活动的时间，可以缩短整个工程的工期
    - 当缩短到一定程度时，关键活动可能会变成非关键活动

7.1查找的基本概念

基本概念

- 查找：在数据集合中寻找满足某种条件的数据元素的过程称为查找
- 查找表（查找结构）：用于查找的数据集合称为查找表，它由同一类型的数据元素（或记录）组成，可以是一个数组或链表等数据类型
- 查找表操作
  - 查询某个特定的数据元素是否在查找表中
  - 检索满足条件的某个特定的数据元素的各种属性
  - 在查找表中插入一个数据元素
  - 从查找表中删除某个数据元素
- 静态查找表：查找后不会对表进行任何修改
- 动态查找表：查找后对表进行修改
- 关键字：数据元素中唯一标识该元素的某个数据项的值，使用基于关键字的查找，查找结果应该是唯一的
- 平均查找长度
  - 在查找过程中，一次查找的长度是指需要比较的关键字次数，而平均查找长度则是所有查找过程中进行关键字的比较次数的平均值
  - 数学表达式 $\mathrm{ASL}=\sum_{i=1}^{n}P_iC_i$
  - 参数含义
    - n是查找表的长度
    - P是查找第i个数据元素的概率,一般认为每个数据元素的查找概率相等,即P=1/n
    - Ci是找到第i个数据元素所需进行的比较次数

平均查找长度是衡量查找算法效率的最主要的指标

7.2顺序查找和折半查找

顺序查找

又称线性查找，主要用于在线性表中进行查找。顺序查找通常分为对一般的无序线性表的顺序查找和对按关键字有序的顺序表的顺序查找

一般线性表的顺序查找

基本思想
- 从线性表的一端开始，逐个检查关键字是否满足 给定的条件。
- 若查找到某个元素的关键字满足给定条件，则查找成功，返回该元素在线性表中的位置
- 若已经查找到表的另一端，但还没有查找到符合给定条件的元素，则返回查找失败的信息

平均查找长度
- 成功
- 失败 $ASL_{不成功}=n+1$

缺点：当n较大时,平均查找长度较大,效率低

优点
- 对数据元素的存储没有要求,顺序存储或链式存储皆可
- 对表中记录的有序性也没有要求,无论记录是否按关键字有序,均可应用

有序表的顺序查找

基本思想
- 假设表L是按关键字从小到大排列的,查找的顺序是从前往后,待查找元素的关键字为 key
- 当查找到第i个元素时,发现第i个元素对应的关键字小于key,但第i+1个元素对应的关键字大于 key,这时就可返回查找失败的信息
- 因为第i个元素之后的元素的关键字均大于key,所以表中不存在关键字为 key的元素

平均查找长度
- 成功 $ASL_{成功}=\sum_{i=1}^{n}P_i(n-i+1)=\frac{n+1}{2}$
- 失败 $ASL_{不成功}=\sum_{j=1}^{n}q_j(l_j-1)=\frac{1+2+\cdots+n+n}{n+1}=\frac{n}{2}+\frac{n}{n+1}$

折半查找

折半查找又称二分查找，它仅适用于有序的顺序表

基本思想
- 首先将给定值key与表中中间位置的元素比较,若相等,则查找成功, 返回该元素的存储位置
- 若不等,则所需查找的元素只能在中间元素以外的前半部分或后半部分
- 然后在缩小的范围内继续进行同样的查找,如此重复,直到找到为止
- 或确定表中没有所需要查找的元素,则查找不成功,返回查找失败的信息

查找过程中可以生成判定树（平衡二叉树）
- 树中每个圆形结点表示一个记录,结点中的值为该记录的关键字值
- 树中最下面的叶结点都是方形的,它表示查找不成功的情况

平均查找长度
- 成功 $ASL=\frac{1}{n}\sum_{i=1}^{n}l_i=\frac{1}{n}(1\times1+2\times2+\cdots+h\times2^{h-1})=\frac{n+1}{n}\log_2(n+1)-1\approx\log_2(n+1)-1$
- 失败

h是树的高度,并且元素个数为n时树高h= $h=\lceil\log_2(n+1)\rceil$

时间复杂度为 $O(\log_2 n)$

仅适合于顺序存储结构,不适合于链式存储结构,且要求元素按关键字有序排列

分块查找

又称索引顺序查找，它吸取了顺序查找和折半查找各自的优点，既有动态结构，又适于快速查找

基本思想
- 将查找表分为若干子块
- 块内的元素可以无序,但块之间是有序的, 第一个块中的最大关键字小于第二个块中的所有记录的关键字，以此类推
- 再建立一个索引表,索引表中的每个元素含有各块的最大关键字和各块中的第一个元素的地址,索引表按关键字有序排列

查找成功长度

将长度为n的查找表均匀地分为b块,每块有s个记录，在等概率情况下
- 索引查找 L_I
- 块内查找的平均查找长度 L_S
- 在块内和索引表中均采用顺序查找,则平均查找长度 $ASL=L_I+L_S=\frac{b+1}{2}+\frac{s+1}{2}=\frac{s^2+2s+n}{2s}$
  - 若s=√n,则平均查找长度取最小值√n + 1
- 若对索引表采用折半查找时,则平均查找长度 $ASL=L_I+L_S=\lceil\log_2(b+1)\rceil+\frac{s+1}{2}$

7.3B树和B+树

B树及其基本操作

概述：又称多路平衡查找树,B树中所有结点的孩子个数的最大值称为B树的阶,通常用m表示

特点
- 树中每个结点至多有m棵子树,即至多含有 m-1个关键字
- 若根结点不是终端结点,则至少有两棵子树
- 除根结点外的所有非叶结点至少有 $\lceil m/2 \rceil$ 棵子树,即至少含有 $\lceil m/2 \rceil - 1$ 个关键字
- 所有的叶结点都出现在同一层次上,并且不带信息
- B树是所有结点的平衡因子均等于0的多路平衡查找树

推导性质
- 结点的孩子个数等于该结点中关键字个数加1
- 如果根结点没有关键字就没有子树,此时 B树为空
- 如果根结点有关键字,则其子树必然大于等于两棵,因为子树个数等于关键字个数加1
- 结点中关键字从左到右递增有序,关键字两侧均有指向子树的指针,左边指针所指子树的所有关键字均小于该关键字,右边指针所指子树的所有关键字均大于该关键字

B树的高度（磁盘存取次数）
- 对任意一棵包含n个关键字、高度为h、阶数为 m的B树
- $h \geq \log_m(n+1)$
- 让每个结点中的关键字个数达到最少,则容纳同样多关键字的B树的高度达到最大

B树的查找
- 在B树上查找到某个结点后，先在有序表中进行查找
- 若找到则查找成功，否则按照对应的指针信息到所指的子树中去查找
- 查找到叶结点时（对应指针为空指针），则说明树中没有对应的关键字，查找失败

B树的插入
- 定位：利用前述的B树查找算法,找出插入该关键字的最低层中的某个非叶结点（注意:插入位置一定是最低层中的某个非叶结点）
- 插入：在B树中,每个非失败结点的关键字个数都在区间 $[\lceil m/2 \rceil - 1, m-1]$
- 插入后的结点关键字个数小于 m,可以直接插入;插入后检查被插入结点内关键字的个数,当插入后的结点关键字个数大于m-1时,必须对结点进行分裂

B树的删除
- 直接删除关键字：若被删除关键字所在结点的关键字个数 $\geq \lceil m/2 \rceil$,表明删除该关键字后仍满足B树的定义,则直接删去该关键字
- 兄弟够借:若被删除关键字所在结点删除前的关键字个数 $= \lceil m/2 \rceil - 1$ 且与此结点相邻的右（或左）兄弟结点的关键字个数 $\geq \lceil m/2 \rceil$ 则需要调整该结点、右(或左)兄弟结点及其双亲结点(父子换位法）,以达到新的平衡
- 兄弟不够借:若被删除关键字所在结点删除前的关键字个数 $= \lceil m/2 \rceil - 1$ 且此时与该结点相邻的左、右兄弟结点的关键字个数均 $= \lceil m/2 \rceil - 1$,则将关键字删除后与左（或右）兄弟结点及双亲结点中的关键字进行合并

B+树的基本概念

特性
- 每个分支结点最多有m棵子树（孩子结点）
- 非叶根结点至少有两棵子树,其他每个分支结点至少有 $\lceil m/2 \rceil$ 棵子树
- 结点的子树个数与关键字个数相等。
- 所有叶结点包含全部关键字及指向相应记录的指针,叶结点中将关键字按大小顺序排列, 并且相邻叶结点按大小顺序相互链接起来
- 所有分支结点（可视为索引的索引）中仅包含它的各个子结点（即下一级的索引块)中关键字的最大值及指向其子结点的指针

M阶的B+树与M阶的B树的主要差异

- 在B+树中,具有n个关键字的结点只含有n棵子树,即每个关键字对应一棵子树;而在B树中,具有n个关键字的结点含有 n+1棵子树
- 在B+树中,每个结点（非根内部结点)的关键字个数n的范围是 $\lceil m/2 \rceil \leq n \leq m$ (根结点:1≤n≤m）;在B树中,每个结点（非根内部结点)的关键字个数n的范围是 $\lceil m/2 \rceil - 1 \leq n \leq m-1$ （根结点:1≤n≤m-1）
- 在B+树中,叶结点包含信息,所有非叶结点仅起索引作用,非叶结点中的每个索引项只含有对应子树的最大关键字和指向该子树的指针,不含有该关键字对应记录的存储地址
- 在B+树中,叶结点包含了全部关键字,即在非叶结点中出现的关键字也会出现在叶结点中;而在B树中,叶结点包含的关键字和其他结点包含的关键字是不重复的

7.4散列表（上）

散列表的基本概念

- 散列函数:一个把查找表中的关键字映射成该关键字对应的地址的函数,记为 Hash(key）=Addr
- 冲突：散列函数可能会把两个或两个以上的不同关键字映射到同一地址
- 散列表:根据关键字而直接进行访问的数据结构
- 对散列表进行查找的时间复杂度为O(1)

散列函数的构造方法

注意

- 散列函数的定义域必须包含全部需要存储的关键字，而值域的范围则依赖于散列表的大小或地址范围
- 散列函数计算出来的地址应该能等概率、均匀地分布在整个地址空间中，从而减少冲突的发生
- 散列函数应尽量简单，能够在较短的时间内计算出任一关键字对应的散列地址

散列函数

直接定址法

- 直接取关键字的某个线性函数值为散列地址
- 散列函数为　　H(key)=key或H(key)=a*key + b　　a和b是常数
- 特点
  - 计算最简单,且不会产生冲突
  - 适合关键字的分布基本连续的情况
  - 若关键字分布不连续,空位较多,则会造成存储空间的浪费

除留余数法

- 假定散列表表长为m,取一个不大于m但最接近或等于m 的质数p,利用以下公式把关键字转换成散列地址
- 散列函数为　　H(key)=key %p
- 特点
  - 最简单、最常用的方法
  - 关键是选好p,使得每个关键字通过该函数转换后等概率地映射到散列空间上的任一地址,从而尽可能减少冲突的可能性

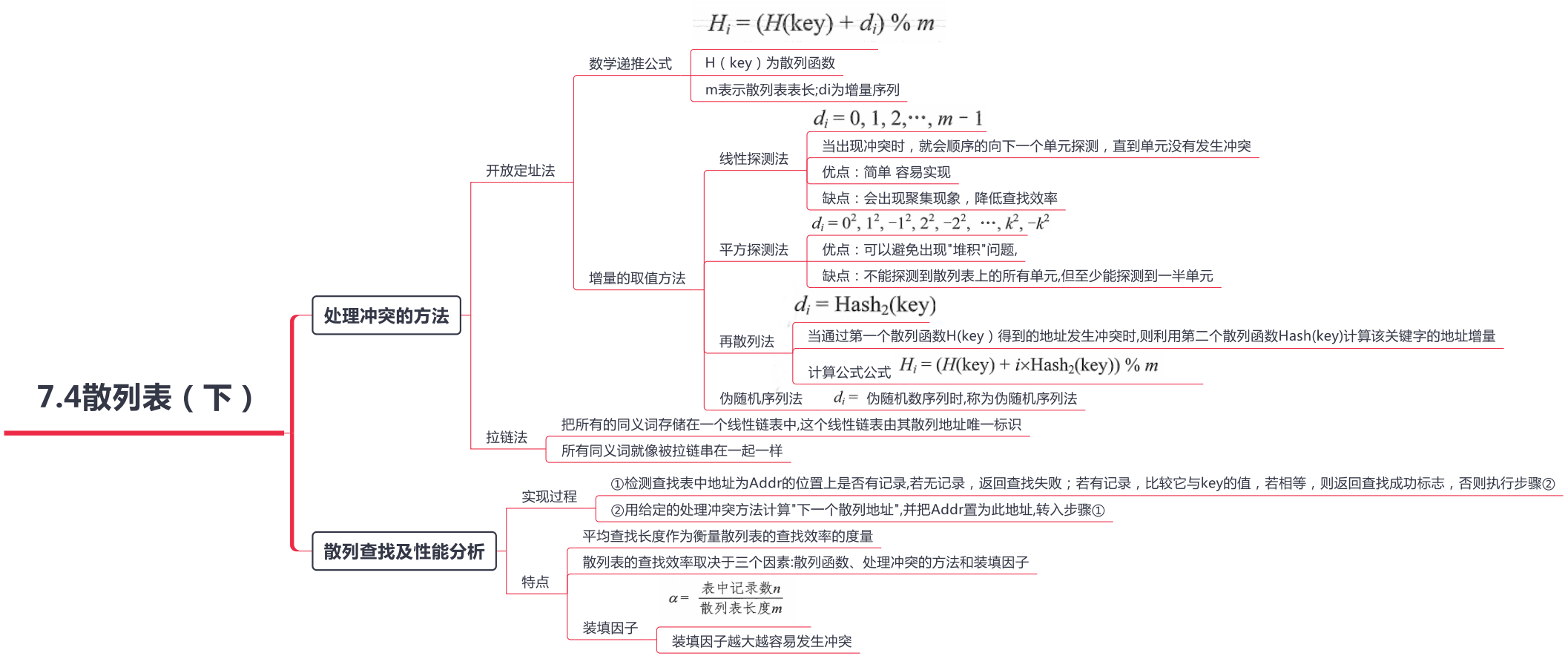
数字分析法

- 设关键字是r进制数,而r个数码在各位上出现的频率不一定相同,可能在某些位上分布均匀一些,每种数码出现的机会均等
- 而在某些位上分布不均匀,只有某几种数码经常出现,此时应选取数码分布较为均匀的若干位作为散列地址
- 特点
  - 适合于已知的关键字集合,若更换了关键字
  - 则需要重新构造新的散列函数

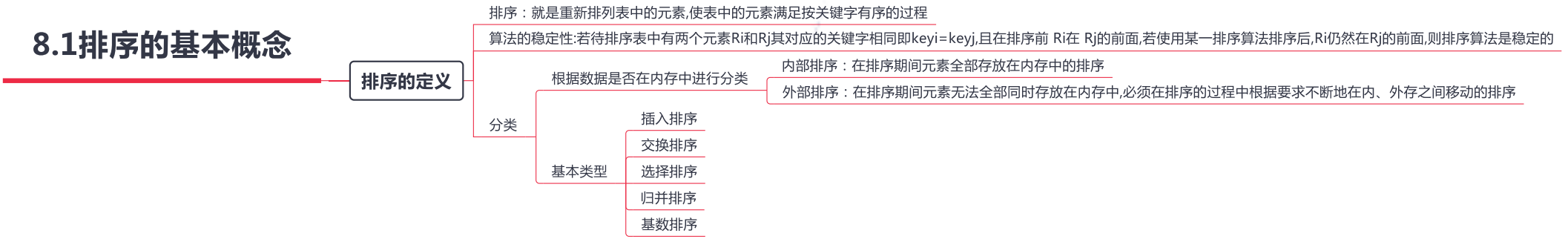
平方取中法

- 取关键字的平方值的中间几位作为散列地址
- 特点
  - 散列地址分布比较均匀
  - 适用于关键字的每位取值都不够均匀或均小于散列地址所需的位数

7.4散列表（下）


处理冲突的方法
开放定址法
数学递推公式
$H_i = (H(\text{key}) + d_i)\ \%\ m$
H（key）为散列函数
m表示散列表表长;di为增量序列
增量的取值方法
线性探测法
$d_i = 0, 1, 2, \cdots, m-1$
当出现冲突时，就会顺序的向下一个单元探测，直到单元没有发生冲突
优点：简单 容易实现
缺点：会出现聚集现象，降低查找效率
平方探测法
$d_i = 0^2, 1^2, -1^2, 2^2, -2^2, \cdots, k^2, -k^2$
优点：可以避免出现"堆积"问题,
缺点：不能探测到散列表上的所有单元,但至少能探测到一半单元
再散列法
$d_i = \text{Hash}_2(\text{key})$
当通过第一个散列函数H(key）得到的地址发生冲突时,则利用第二个散列函数Hash(key)计算该关键字的地址增量
计算公式公式 $H_i = (H(\text{key}) + i \times \text{Hash}_2(\text{key}))\ \%\ m$
伪随机序列法
$d_i =$ 伪随机数序列时,称为伪随机序列法
拉链法
把所有的同义词存储在一个线性链表中,这个线性链表由其散列地址唯一标识
所有同义词就像被拉链串在一起一样
散列查找及性能分析
实现过程
①检测查找表中地址为Addr的位置上是否有记录,若无记录，返回查找失败；若有记录，比较它与key的值，若相等，则返回查找成功标志，否则执行步骤②
②用给定的处理冲突方法计算"下一个散列地址",并把Addr置为此地址,转入步骤①
特点
平均查找长度作为衡量散列表的查找效率的度量
散列表的查找效率取决于三个因素:散列函数、处理冲突的方法和装填因子
$\alpha = \frac{\text{表中记录数}n}{\text{散列表长度}m}$
装填因子
装填因子越大越容易发生冲突

8.1排序的基本概念


排序的定义
排序：就是重新排列表中的元素,使表中的元素满足按关键字有序的过程
算法的稳定性:若待排序表中有两个元素Ri和Rj其对应的关键字相同即keyi=keyj,且在排序前 Ri在 Rj的前面,若使用某一排序算法排序后,Ri仍然在Rj的前面,则排序算法是稳定的
分类
根据数据是否在内存中进行分类
内部排序：在排序期间元素全部存放在内存中的排序
外部排序：在排序期间元素无法全部同时存放在内存中,必须在排序的过程中根据要求不断地在内、外存之间移动的排序
基本类型
插入排序
交换排序
选择排序
归并排序
基数排序

8.2 插入排序


概述
基本思想
每次将一个待排序的记录按其关键字大小插入到前面已排好序的子序列中,直到全部记录插入完成
引申出的算法
直接插入排序
折半插入排序
希尔排序
直接插入排序
实现过程
将整个序列分为两部分
有序序列
无序序列
每次从无序序列中取出一个元素，然后在有序序列中遍历，寻找合适的位置将该元素插入有序序列中
自插入的位置开始有序序列向后移动一个元素位置
性能分析
空间效率:空间复杂度为O(1)
时间效率
最好情况
表中元素已经有序,此时每插入一个元素,都只需比较一次而不用移动元素
时间复杂度为 O(n)
最坏情况
表中元素顺序刚好与排序结果中的元素顺序相反（逆序）
总的比较次数达到最大 $\sum_{i=2}^{n} i$
总的移动次数也达到最大 $\sum_{i=2}^{n}(i+1)$
平均情况
时间复杂度 $O(n^2)$
时间复杂度为 $O(n^2)$
稳定算法
适用于 顺序存储和链式存储
折半插入排序
实现过程
首先确定折半插入排序的范围
然后对其进行类似于二分法定界的方式，不断缩小其范围
最后对数据进行移动，对待排序算法进行插入
性能分析
空间效率：空间复杂度为O(1)
时间效率
比较次数为 $O(n\log_2 n)$
该算法比较次数仅与表中元素个数有关
稳定算法
适用于顺序存储
希尔排序
实现过程
先将待排序表分割成若干形如 L[i, i + d, i + 2d,..., i + kd] 的“特殊”子表，对各个子表分别进行直接插入排序。缩小增量d，重复上述过程，直到d=1为止
每次排序去一定的步长，然后在选定的元素中进行直接插入排序
步长逐渐减小，最后为1
性能分析
空间效率:空间复杂度为O(1)
时间复杂度为 $O(n^2)$
不稳定算法
适用于顺序存储

8.3 交换排序

- **基本概述**
  - 根据序列中两个元素关键字的比较结果来对换这两个记录在序列中的位置
  - 考试涉及范围
    - 冒泡排序
    - 快速排序
- **冒泡排序**
  - 基本思想
    - 从后往前（或从前往后)两两比较相邻元素的值,若为逆序（即A[i-1]>A[i]）,则交换它们,直到序列比较完
    - 将最小的元素交换到待排序列的第一个位置(或将最大的元素交换到待排序列的最后一个位置)
    - 进行下一趟冒泡时,前一趟确定的最小元素不再参与比较,每趟冒泡的结果是把序列中的最小元素(或最大元素）放到了序列的最终位置
    - 如果某一趟排序过程中未发生"交换"，则算法可提前结束
  - 性能分析
    - 空间复杂度：O(1)
    - 时间效率
      - 最好情况　时间复杂度为O(n)
      - 最坏情况　时间复杂度 $O(n^2)$
      - 平均情况
    - 冒泡排序时间复杂度为 $O(n^2)$
  - 稳定算法
  - 适用于顺序存储
- **快速排序被认为是目前基于比较的内部排序方法中最好的方法**
- **快速排序**
  - 基本思想
    - 首先选取一个元素作为枢纽，然后以此枢轴为界分为两个部分，左面小于该枢轴值，右面大于该枢轴值
    - 然后再对这两个部分分别递归的进行上述步骤
  - 性能分析
    - 空间复杂度
      - 最好情况 $O(\log_2 n)$
      - 最坏情况　O(n)
      - 平均情况 $O(\log_2 n)$
    - 时间效率
      - 快速排序的运行时间与划分是否对称有关
      - 对应于初始排序表基本有序或基本逆序时,就得到最坏情况下的时间复杂度为O(n^2）
    - 快速排序的时间复杂度为 $O(n^2)$
  - 不稳定算法
  - 适用于顺序存储

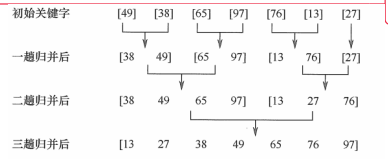
8.4选择排序

- **基本思想**
  - 每一趟（如第i趟）在后面n-i+1（i=1,2.....n-1）个待排序元素中选取关键字最小的元素,作为有序子序列的第i个元素,直到第n-1趟做完,待排序元素只剩下1个,就不用再选
- **简单选择排序**
  - 基本思想
    - 将表分为两部分，有序部分和无序部分
    - 每次从无序部分中选取最小的元素，然后将其放入有序部分中
  - 性能分析
    - 空间效率:O(1)
    - 时间效率
      - 元素间比较的次数与序列的初始状态无关
      - 时间复杂度为 $O(n^2)$
  - 不稳定的排序算法
  - 适合顺序存储
- **堆排序**
  - 基本概述
    - 大根堆：父节点的值大于相对应的孩子结点值
    - 小跟堆：父节点的值小于相对应的孩子结点值
  - 基本思想
    - 建堆：按照大根堆或者小根堆的规则建立起相应的二叉树，那么根节点一定是最大值或者最小值
    - 调整堆：当根节点输出后，整颗二叉树可能被破坏，这是要根据相应的建堆规则，自底向上，自左向右，进行父节点与子节点交换以满足相应的建堆规则
  - 性能分析
    - 空间效率:空间复杂度为O(1)
    - 时间效率　建堆时间为O(n)　调整的时间复杂度为O(h)　h 为二叉树的高度
    - 时间复杂度 $O(n\log_2 n)$
  - 不稳定排序算法

8.5 归并排序和基数排序

归并排序

- 基本思想
  - 每次选定相应的元素分别合成一个新的有序表
    - 2路归并——二合一
    - k路归并——k合一
  - 示意图



- 性能分析
  - 空间复杂度为O(n)
  - 时间复杂度为 $O(n\log_2 n)$
- 稳定排序算法
- 适用于顺序表

基数排序

- 排序思想
  - 最高位优先（MSD）法：按关键字位权重递减依次逐层划分成若干更小的子序列,最后将所有子序列依次连接成一个有序序列
  - 最低位优先（LSD）法：按关键字权重递增依次进行排序,最后形成一个有序序列
- 性能分析
  - 空间效率
    - 一趟排序需要的辅助存储空间为r(r个队列:r个队头指针和r个队尾指针）
    - 基数排序的空间复杂度为O(r）
  - 时间效率
    - 基数排序需要进行d趟分配和收集,一趟分配需要O(n）,一趟收集需要O(r)
    - 基数排序的时间复杂度为O(d(n+r)）　　与序列的初始状态无关
- 稳定排序算法

算法种类	时间复杂度			空间复杂度	是否稳定
	最好情况	平均情况	最坏情况		
直接插入排序	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	是
冒泡排序	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	是
简单选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	否
希尔排序				$O(1)$	否
快速排序	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n^2)$	$O(\log_2 n)$	否
堆排序	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(1)$	否
2 路归并排序	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n)$	是
基数排序	$O(d(n+r))$	$O(d(n+r))$	$O(d(n+r))$	$O(r)$	是

8.6各种内部排序算法的比较及应用

排序算法小结

- 若n较小,可采用直接插入排序或简单选择排序
- 当记录本身信息量较大时,用简单选择排序较好
- 若文件的初始状态已按关键字基本有序,则选用直接插入或冒泡排序为宜
- 快速排序被认为是目前基于比较的内部排序方法中最好的方法　　待排序的关键字随机分布时,快速排序的平均时间最短
- 若n较大,则应采用时间复杂度为 $O(n\log_2 n)$ 的排序方法:快速排序、堆排序或归并排序
- 要求排序稳定且时间复杂度为 $O(n\log_2 n)$ 则可选用归并排序
- 若n很大,记录的关键字位数较少且可以分解时,采用基数排序较好
- 当记录本身信息量较大时,为避免耗费大量时间移动记录,可用链表作为存储结构

8.7 外部排序（上）

- **外部排序的基本概念**
  - 对大文件进行排序,因为文件中的记录很多、信息量庞大,无法将整个文件复制进内存中进行排序
  - 需要将待排序的记录存储在外存上,排序时再把数据一部分一部分地调入内存进行排序,在排序过程中需要多次进行内存和外存之间的交换
- **外部排序的方法**
  - 基本概念
    - 文件通常是按块存储在磁盘上的,操作系统也是按块对磁盘上的信息进行读写的
    - 外部排序过程中的时间代价主要考虑访问磁盘的次数,即 I/O次数
  - 外部排序通常采用归并排序法　　算法实现的两个阶段
    - 据内存缓冲区大小,将外存上的文件分成若干长度为f的子文件,依次读入内存并利用内部排序方法对它们进行排序, 并将排序后得到的有序子文件重新写回外存（归并段或顺串）
    - 对这些归并段进行逐趟归并,使归并段（有序子文件）逐渐由小到大,直至得到整个有序文件为止
  - 耗费时间　　外部排序的总时间= 内部排序所需的时间 + 外存信息读写的时间 + 内部归并所需的时间
  - 归并排序优化
    - 增大归并路数k
    - 减少初始归并段个数r

    都能减少归并趟数s,进而减少读写磁盘的次数,达到提高外部排序速度的目的
- **多路平衡归并与败者树**
  - 引入败者树的背景　　为了使内部归并不受k（归并路数）的增大的影响
  - 基本思想
    - 败者树是树形选择排序的一种变体, 可视为一棵完全二叉树
    - k个叶结点分别存放k个归并段在归并过程中当前参加比较的记录,内部结点用来记忆左右子树中的"失败者",而让胜者往上继续进行比较,一直到根结点
    - 若比较两个数,大的为失败者、小的为胜利者,则根结点指向的数为最小数
  - 性能分析
    - k 路归并的败者树深度 $\lceil \log_2 k \rceil$
    - 总的比较次数 $S(n-1)\lceil \log_2 k \rceil = \lceil \log_k r \rceil (n-1)\lceil \log_2 k \rceil = (n-1)\lceil \log_2 r \rceil$
  - 注意
    - 归并路数k并不是越大越好。归并路数k增大时,相应地需要增加输入缓冲区的个数
    - 当k值过大时,虽然归并趟数会减少,但读写外存的次数仍会增加
  - 优化
    - 增加归并路数 k，进行多路平衡归并
      - 代价1：需要增加相应的输入缓冲区
      - 代价2：每次从k个归并段中选一个最小元素需要（k-1）次关键字对比
    - 减少初始归并段数量r

8.7 外部排序（下）

置换-选择排序（生成初始归并段）

实现过程

- 设初始待排文件为FI,初始归并段输出文件为FO,内存工作区为WA,FO和WA的初始状态为空,WA 可容纳 w个记录
- 1）从FI输入 w个记录到工作区 WA
- 2）从 WA中选出其中关键字取最小值的记录,记为MINIMAX记录
- 3）将 MINIMAX 记录输出到 FO中去
- 4）若FI不空,则从FI输入下一个记录到WA中
- 5)从 WA中所有关键字比MINIMAX 记录的关键字大的记录中选出最小关键字记录,作为新的 MINIMAX记录
- 6）重复3）~5）,直至在WA中选不出新的MINIMAX记录为止,由此得到一个初始归并段,输出一个归并段的结束标志到 FO中去
- 7）重复2）~6）,直至WA为空。由此得到全部初始归并段

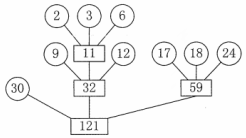
最佳归并树

结构概述

- 各叶结点表示一个初始归并段,上面的权值表示该归并段的长度
- 叶结点到根的路径长度表示其参加归并的趟数
- 各非叶结点代表归并成的新归并段
- 根结点表示最终生成的归并段
- 树的带权路径长度WPL为归并过程中的总读记录数

算法优化

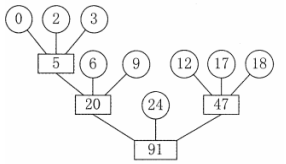
- 引入哈夫曼树的思想
- 在归并树中,让记录数少的初始归并段最先归并,记录数多的初始归并段最晚归并,就可以建立总的I/O次数最少的最佳归并树



示意图

算法修正

- 若初始归并段不足以构成一棵严格 k 叉树时,需添加长度为0的"虚段"
- 按照哈夫曼树的原则,权为0的叶子应离树根最远



示意图

需要修正的条件

- 设度为0的结点有 n_0（$=n$）个,度为k的结点有 n_k 个
- 严格k叉树有 $n_0=(k-1)n_k+1$ 变形可得 $n_k=(n_0-1)/(k-1)$
- $(n_0-1)\%(k-1)=0$ 说明正好可以构造k叉归并树
- $(n_0-1)\%(k-1)=u\neq 0$ 再加上k-u-1个空归并段,就可以建立归并树